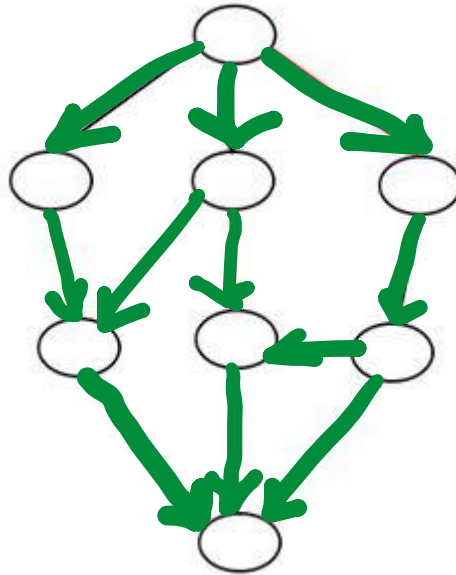


Максимальный
поток,
минимальный
разрез,
паросочетания
ИТМО ИС

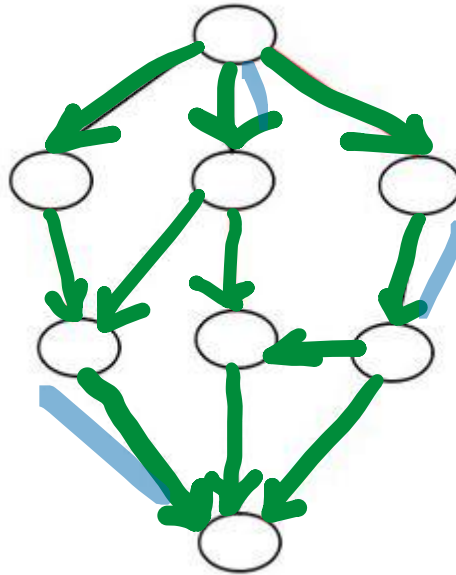
Задача о максимальном паросочетании

Паросочетание (англ. matching) M – произвольное множество рёбер графа такое, что никакие два ребра не имеют общей вершины.



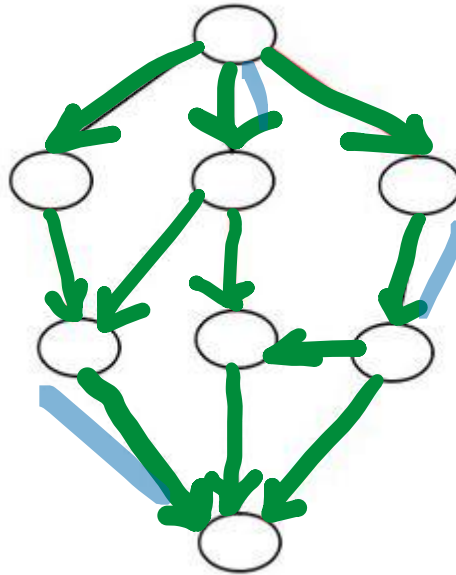
Задача о максимальном паросочетании

Паросочетание (англ. matching) M – произвольное множество рёбер графа такое, что никакие два ребра не имеют общей вершины.



Задача о максимальном паросочетании

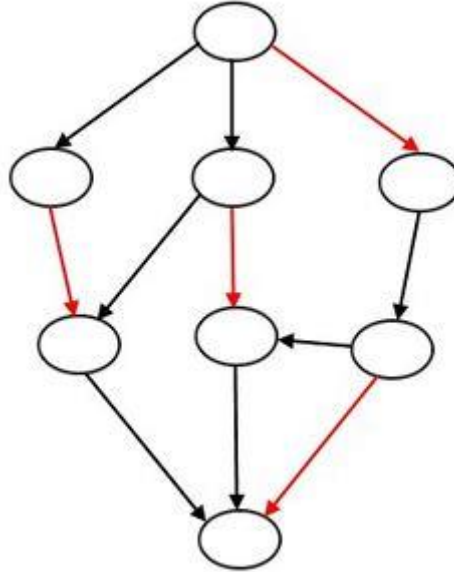
Паросочетание (англ. matching) M – произвольное множество рёбер графа такое, что никакие два ребра не имеют общей вершины.



можно
ли
лучше?

Задача о максимальном паросочетании

Паросочетание (англ. matching) M – произвольное множество рёбер графа такое, что никакие два ребра не имеют общей вершины.



ga!

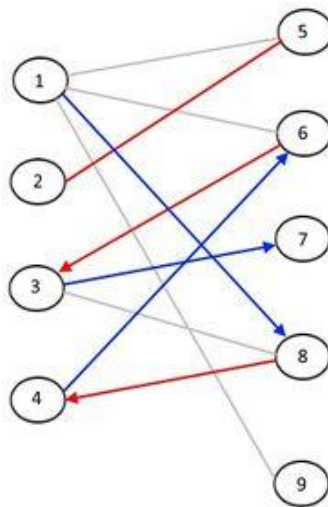
Задача о максимальном паросочетании

Определение:

Чередующаяся цепь (англ. *alternating path*) — путь в двудольном графе, для любых двух соседних рёбер которого верно, что одно из них принадлежит паросочетанию M , а другое нет.



Чередующаяся цепь:
1-8-4-6-3-7



Задача о максимальном паросочетании

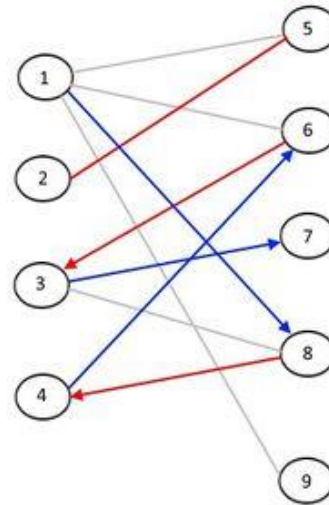
Определение:

Чередующаяся цепь (англ. *alternating path*) — путь в двудольном графе, для любых двух соседних рёбер которого верно, что одно из них принадлежит паросочетанию M , а другое нет.



Начинается и заканчивается
черной линией

Чередующаяся цепь:
1-8-4-6-3-7



Задача о максимальном паросочетании

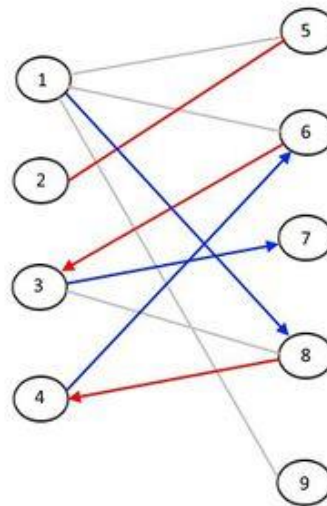
Определение:

Чередующаяся цепь (англ. *alternating path*) — путь в двудольном графе, для любых двух соседних рёбер которого верно, что одно из них принадлежит паросочетанию M , а другое нет.



как еще может быть?

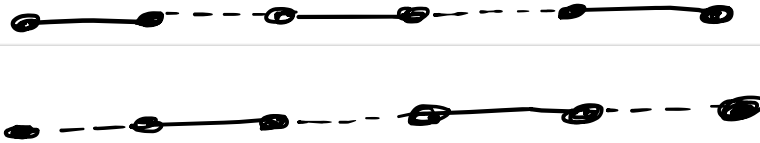
Чередующаяся цепь:
1-8-4-6-3-7



Задача о максимальном паросочетании

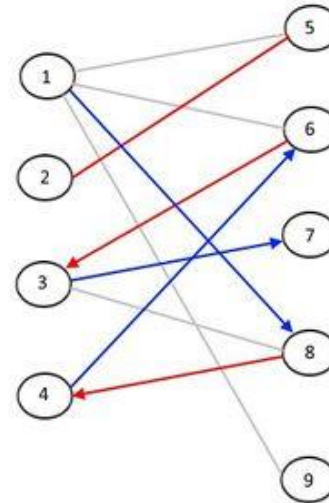
Определение:

Чередующаяся цепь (англ. *alternating path*) — путь в двудольном графе, для любых двух соседних рёбер которого верно, что одно из них принадлежит паросочетанию M , а другое нет.



Начинается и заканчивается свободным ребром

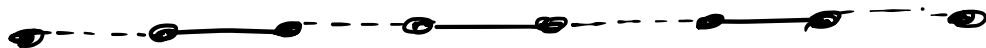
Чередующаяся цепь:
1-8-4-6-3-7



Задача о максимальном паросочетании

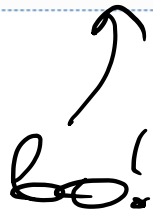
Определение:

Чередующаяся цепь (англ. *alternating path*) — путь в двудольном графе, для любых двух соседних рёбер которого верно, что одно из них принадлежит паросочетанию M , а другое нет.

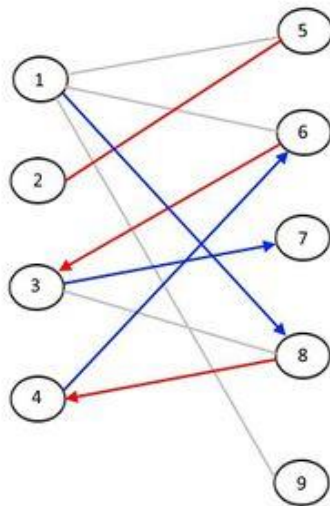


Определение:

Дополняющая цепь (или увеличивающая цепь) (англ. *augmenting path*) — чередующаяся цепь, у которой оба конца свободны.



Чередующаяся цепь:
1-8-4-6-3-7



Задача о максимальном паросочетании

Определение:

Чередующаяся цепь (англ. *alternating path*) — путь в двудольном графе, для любых двух соседних рёбер которого верно, что одно из них принадлежит паросочетанию M , а другое нет.



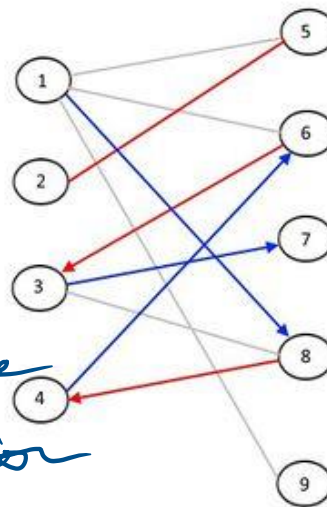
Определение:

Дополняющая цепь (или увеличивающая цепь) (англ. *augmenting path*) — чередующаяся цепь, у которой оба конца свободны.

60!

Чередующаяся цепь:
1-8-4-6-3-7

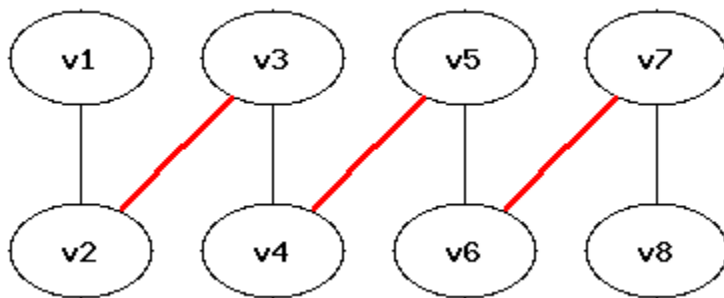
Свободные
вершины



Задача о максимальном паросочетании

Теорема:

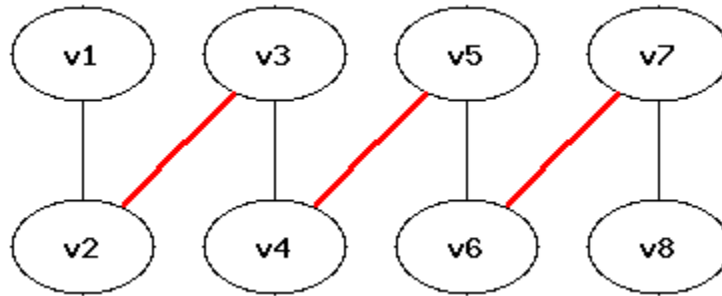
Паросочетание M в двудольном графе G является максимальным тогда и только тогда, когда в G нет дополняющей цепи.



Задача о максимальном паросочетании

Теорема:

Паросочетание M в двудольном графе G является максимальным тогда и только тогда, когда в G нет дополняющей цепи.



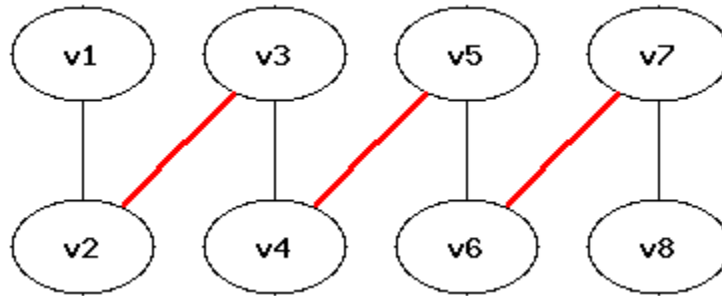
идея доказательства?

Задача о максимальном паросочетании

Теорема:

Паросочетание M в двудольном графе G является максимальным тогда и только тогда, когда в G нет дополняющей цепи.

красные — берем
черные — нет

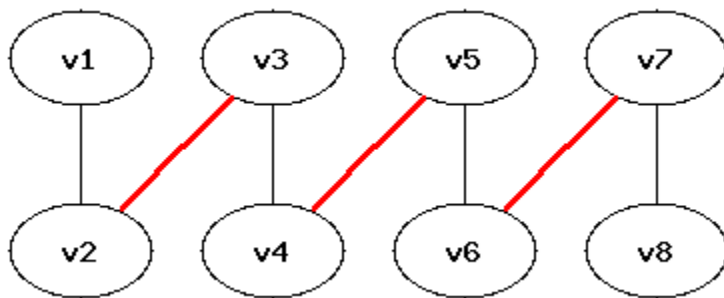


Задача о максимальном паросочетании

Теорема:

Паросочетание M в двудольном графе G является максимальным тогда и только тогда, когда в G нет дополняющей цепи.

есть $M_{\max}$ 
 M' — мы
нашли



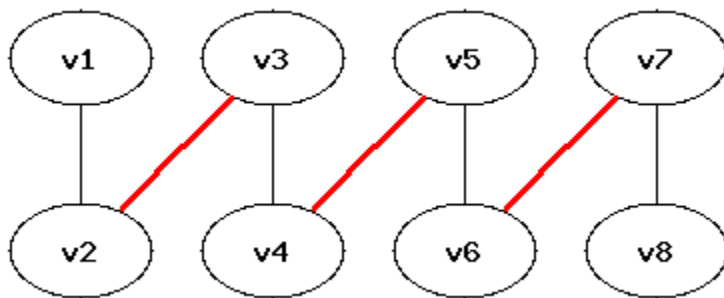
Пусть есть два паросочетания. Фиолетовое – настоящее максимальное, а зеленое – это то, что мы нашли

Задача о максимальном паросочетании

Теорема:

Паросочетание M в двудольном графе G является максимальным тогда и только тогда, когда в G нет дополняющей цепи.

есть $M_{\max}$
 M' — мы
нашли



Уберем
остальные
ребра

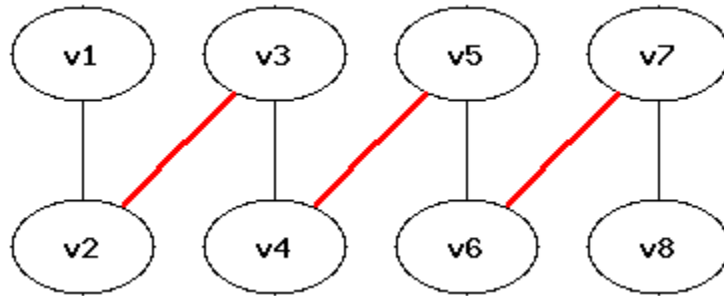
Задача о максимальном паросочетании

Теорема:

Паросочетание M в двудольном графе G является максимальным тогда и только тогда, когда в G нет дополняющей цепи.

есть $M_{\max}$

M' — мы
нашли



уберем
остальные

сколько выходит
ребер из вершины?

Задача о максимальном паросочетании

Теорема:

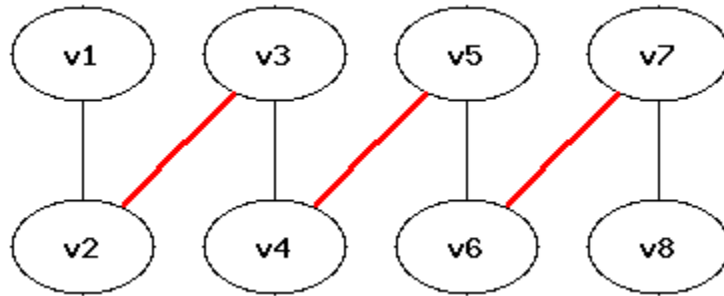
Паросочетание M в двудольном графе G является максимальным тогда и только тогда, когда в G нет дополняющей цепи.

есть $M_{\max}$

M' — мы
нашли

$\leq 2!$

сколько выходов
ребер из вершины?

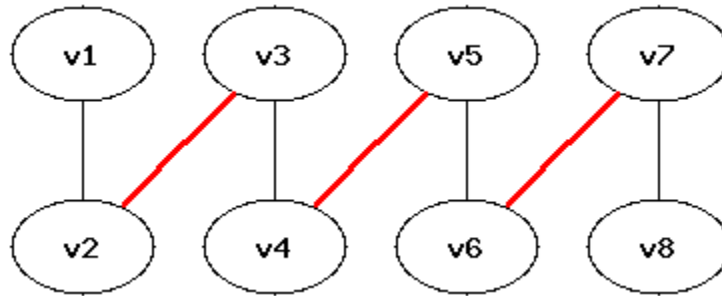


уберем
остальные

Задача о максимальном паросочетании

Теорема:

Паросочетание M в двудольном графе G является максимальным тогда и только тогда, когда в G нет дополняющей цепи.



что
⇒ то
тогда?

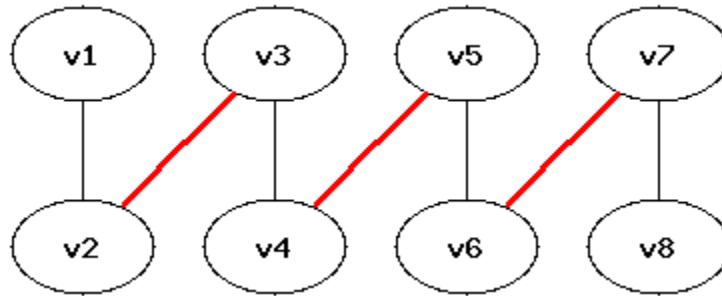


$\leq 2!$

Задача о максимальном паросочетании

Теорема:

Паросочетание M в двудольном графе G является максимальным тогда и только тогда, когда в G нет дополняющей цепи.



что
→ то
тогда?



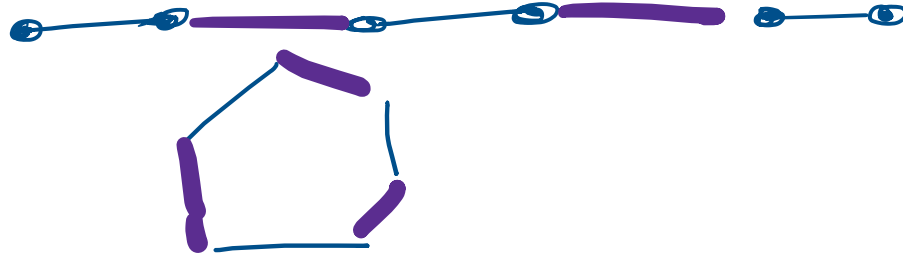
$\leq 2!$

генерация/исключения

Задача о максимальном паросочетании

Теорема:

Паросочетание M в двудольном графе G является максимальным тогда и только тогда, когда в G нет дополняющей цепи.



что
→ то
тогда?



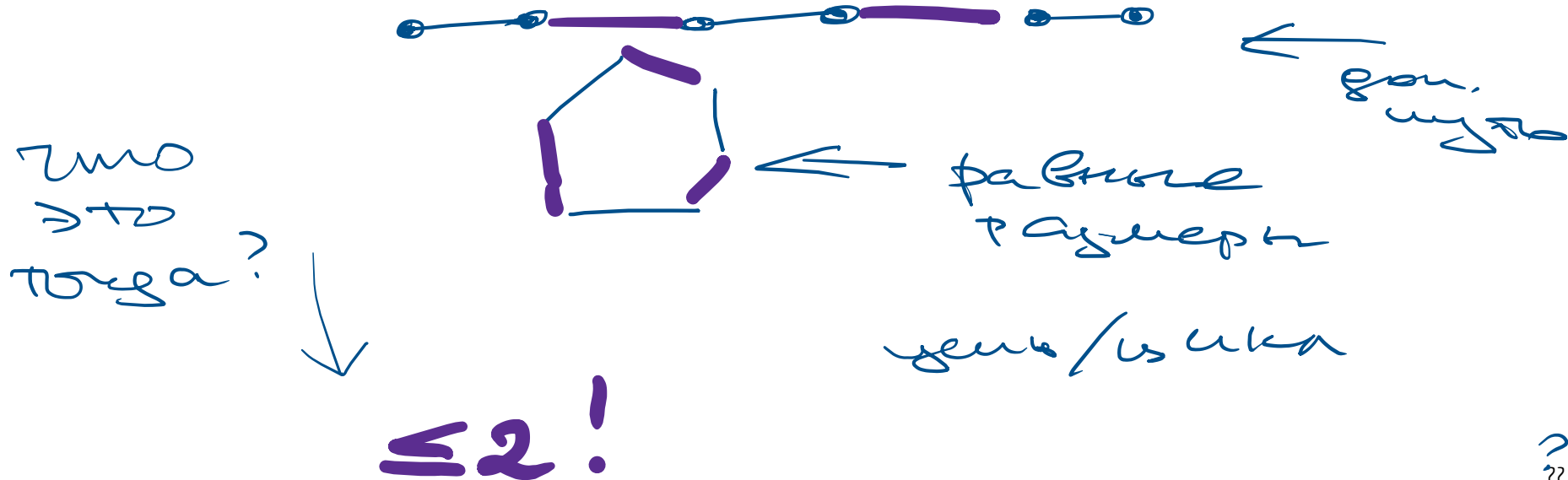
$\leq 2!$

генерация

Задача о максимальном паросочетании

Теорема:

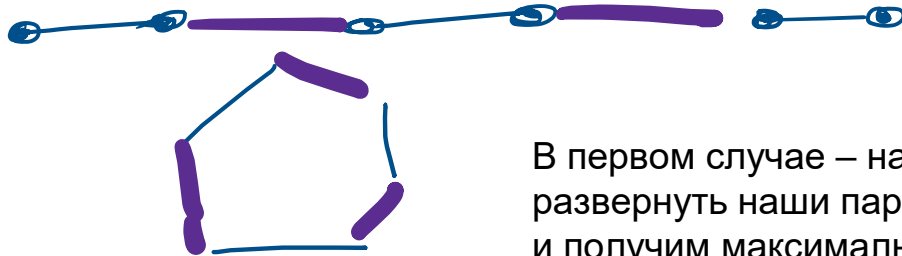
Паросочетание M в двудольном графе G является максимальным тогда и только тогда, когда в G нет дополняющей цепи.



Задача о максимальном паросочетании

Теорема:

Паросочетание M в двудольном графе G является максимальным тогда и только тогда, когда в G нет дополняющей цепи.



что
→ то
тогда?



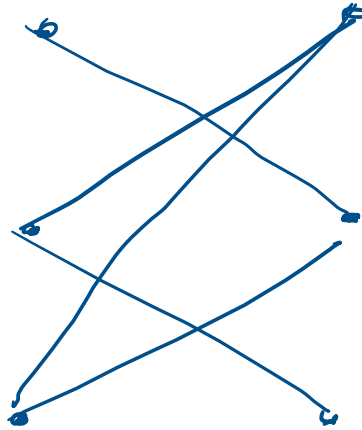
$\leq 2!$

В первом случае – надо просто развернуть наши паросочетания M, M' и получим максимальное
Во втором случае паросочетания имеют одинаковый размер

Задача о максимальном паросочетании. Двудольный граф. Алгоритм Куна

Теорема:

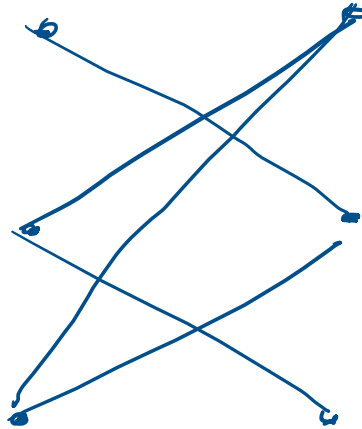
Паросочетание M в двудольном графе G является максимальным тогда и только тогда, когда в G нет дополняющей цепи.



Задача о максимальном паросочетании. Двудольный граф. Алгоритм Куна

Теорема:

Паросочетание M в двудольном графе G является максимальным тогда и только тогда, когда в G нет дополняющей цепи.



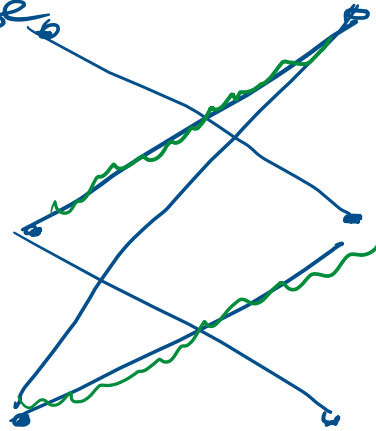
Давайте сделаем ориентацию ребер

Задача о максимальном паросочетании. Двудольный граф. Алгоритм Куна

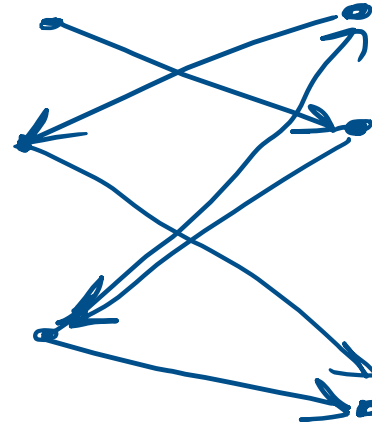
Теорема:

Паросочетание M в двудольном графе G является максимальным тогда и только тогда, когда в G нет дополняющей цепи.

ориент. вправо = свободное
влево = занятое



ребро



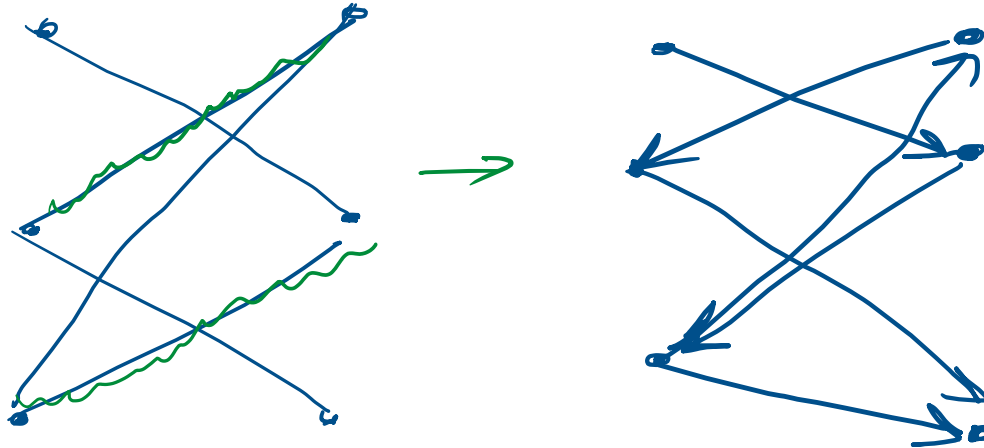
Задача о максимальном паросочетании. Двудольный граф. Алгоритм Куна

Теорема:

Паросочетание M в двудольном графе G является максимальным тогда и только тогда, когда в G нет дополняющей цепи.

Пусть тут наше паросочетание выделено зеленым

Тогда ориентация ребер показана на картинке справа



Задача о максимальном паросочетании. Двудольный граф. Алгоритм Куна

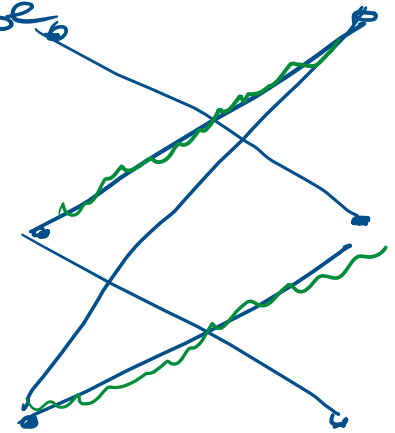
Теорема:

Паросочетание M в двудольном графе G является максимальным тогда и только тогда, когда в G нет дополняющей цепи.

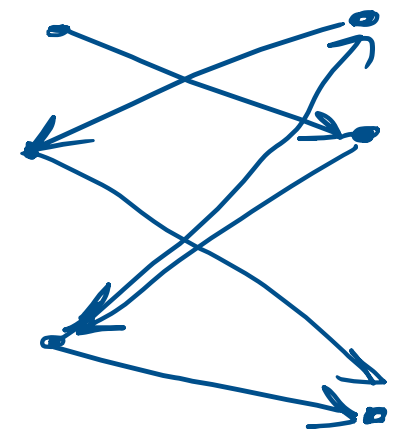
ориент. вправо = свободное

влево = занятое

доп. путь?



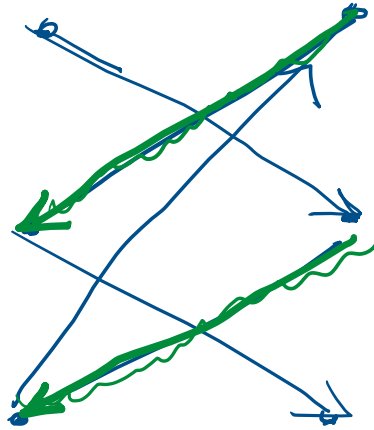
ребро



Задача о максимальном паросочетании. Двудольный граф. Алгоритм Куна

Теорема:

Паросочетание M в двудольном графе G является максимальным тогда и только тогда, когда в G нет дополняющей цепи.



путь чередует
своб./зан.
вершины

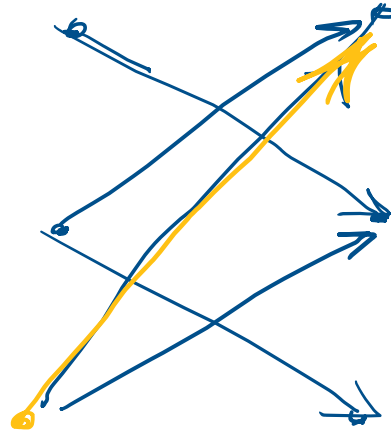
Задача о максимальном паросочетании.

Двудольный граф. Алгоритм Куна

Теорема:

Паросочетание M в двудольном графе G является максимальным тогда и только тогда, когда в G нет дополняющей цепи.

Стартуем из
случайной
вершины
левой доли.
Все ребра
повернуты
вправо



путь чередует
своб./зан.
вершины

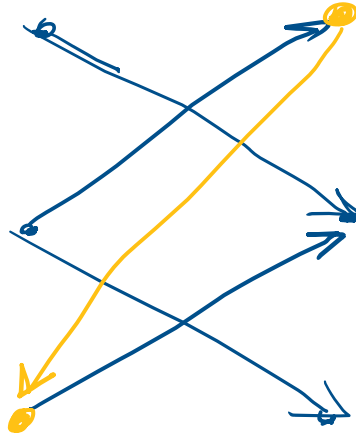
Задача о максимальном паросочетании.

Двудольный граф. Алгоритм Куна

Теорема:

Паросочетание M в двудольном графе G является максимальным тогда и только тогда, когда в G нет дополняющей цепи.

Разворачиваем ребра на
выбранном пути. Теперь длина
парсоч == 1



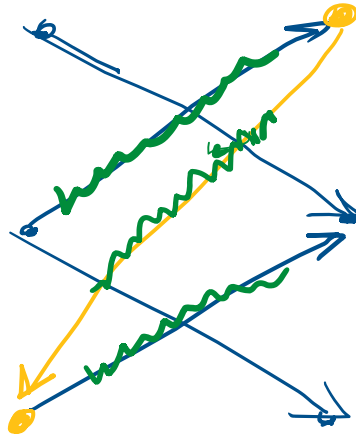
путь чередует
своб./зан.
вершины

Задача о максимальном паросочетании. Двудольный граф. Алгоритм Куна

Теорема:

Паросочетание M в двудольном графе G является максимальным тогда и только тогда, когда в G нет дополняющей цепи.

Опять стартуем из левой доли. Наш путь длины 3



путь чередует
своб./зан.
вершины

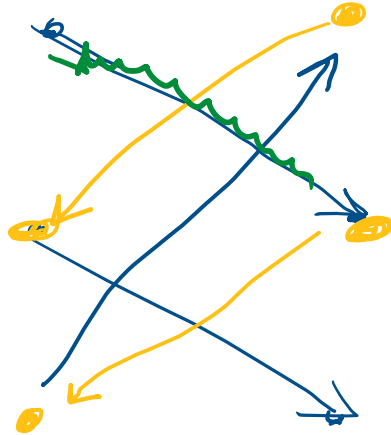
Задача о максимальном паросочетании.

Двудольный граф. Алгоритм Куна

Теорема:

Паросочетание M в двудольном графе G является максимальным тогда и только тогда, когда в G нет дополняющей цепи.

Произвели разворот пути и теперь
начали из левой верхней вершины

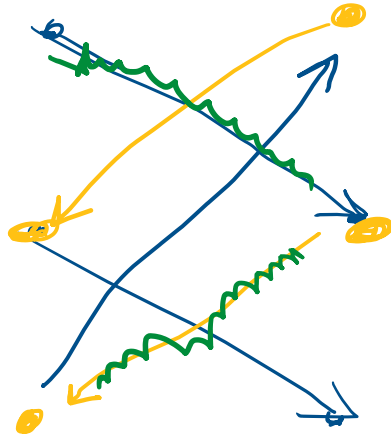


путь чередует
своб./зан.
вершины

Задача о максимальном паросочетании. Двудольный граф

Теорема:

Паросочетание M в двудольном графе G является максимальным тогда и только тогда, когда в G нет дополняющей цепи.

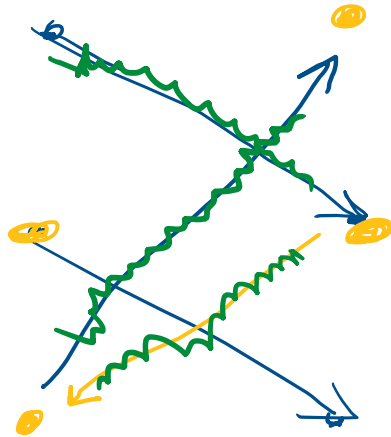


путь чередует
своб./зан.
вершины

Задача о максимальном паросочетании. Двудольный граф

Теорема:

Паросочетание M в двудольном графе G является максимальным тогда и только тогда, когда в G нет дополняющей цепи.

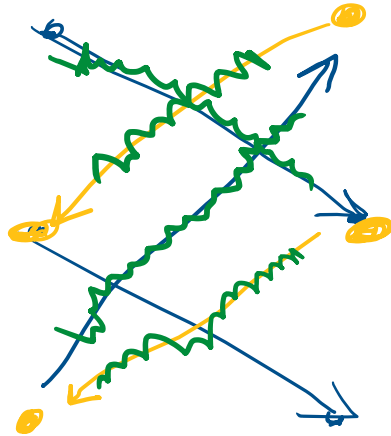


путь чередует
своб./зан.
вершины

Задача о максимальном паросочетании. Двудольный граф

Теорема:

Паросочетание M в двудольном графе G является максимальным тогда и только тогда, когда в G нет дополняющей цепи.



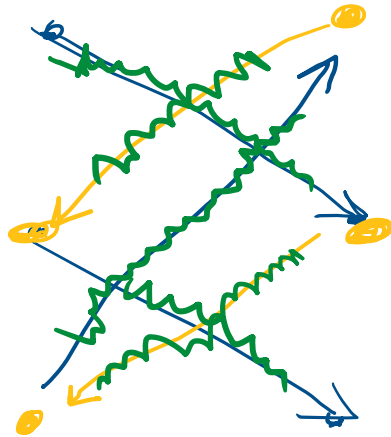
путь чередует
своб. / зам.
вершины

Задача о максимальном паросочетании.

Двудольный граф

Теорема:

Паросочетание M в двудольном графе G является максимальным тогда и только тогда, когда в G нет дополняющей цепи.



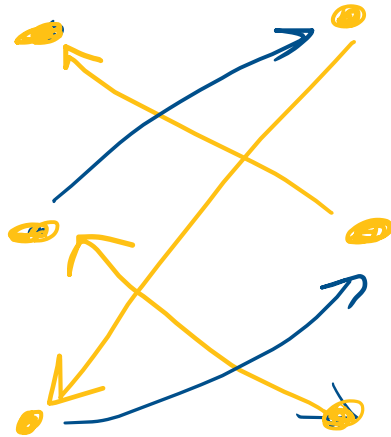
путь чередует
своб./зан.
вершины

Задача о максимальном паросочетании.

Двудольный граф

Теорема:

Паросочетание M в двудольном графе G является максимальным тогда и только тогда, когда в G нет дополняющей цепи.



путь чередует
своб./зан.
вершины

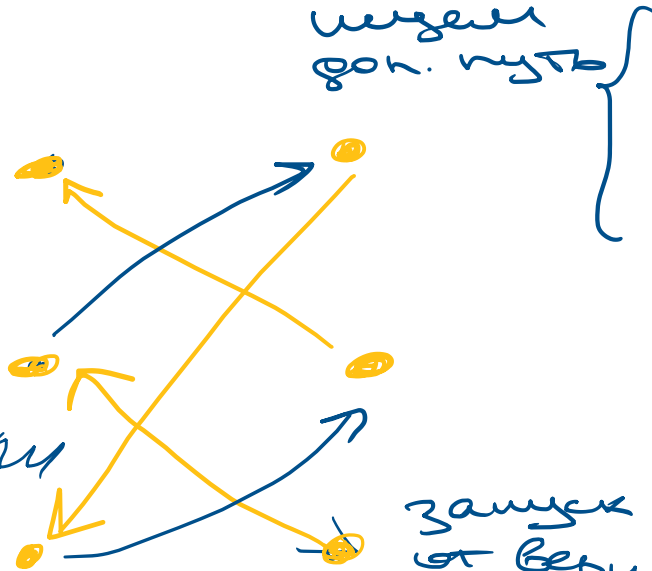
Задача о максимальном паросочетании.

Двудольный граф

Теорема:

Паросочетание M в двудольном графе G является максимальным тогда и только тогда, когда в G нет дополняющей цепи.

если
то
нет-закончили



```
bool dfs(v: int):  
    if (used[v])  
        return false  
    used[v] = true  
    for to in g[v]  
        if (matching[to] == -1 or dfs(matching[to])):  
            matching[to] = v  
            return true  
    return false
```

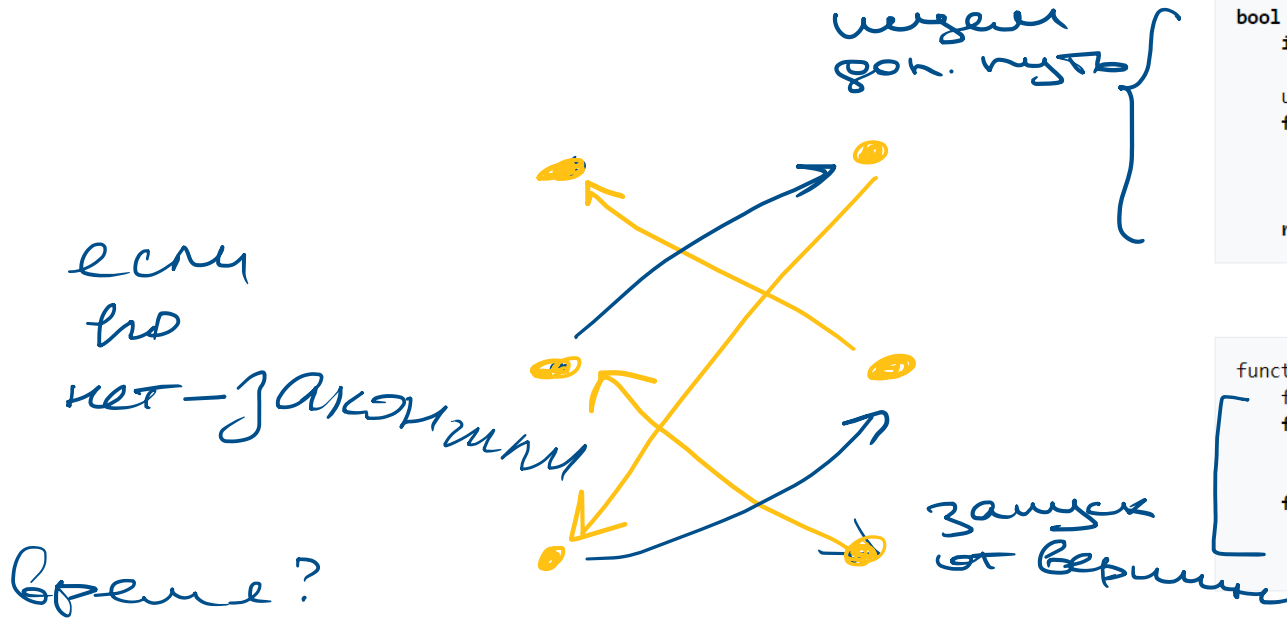
```
function main():  
    fill(matching, -1)  
    for i = 1..n  
        fill(used, false)  
        dfs(i)  
    for i = 1..n  
        if (matching[i] != -1)  
            print(i, " ", matching[i])
```

Задача о максимальном паросочетании.

Двудольный граф

Теорема:

Паросочетание M в двудольном графе G является максимальным тогда и только тогда, когда в G нет дополняющей цепи.



```
bool dfs(v: int):  
    if (used[v])  
        return false  
    used[v] = true  
    for to in g[v]  
        if (matching[to] == -1 or dfs(matching[to])):  
            matching[to] = v  
            return true  
    return false
```

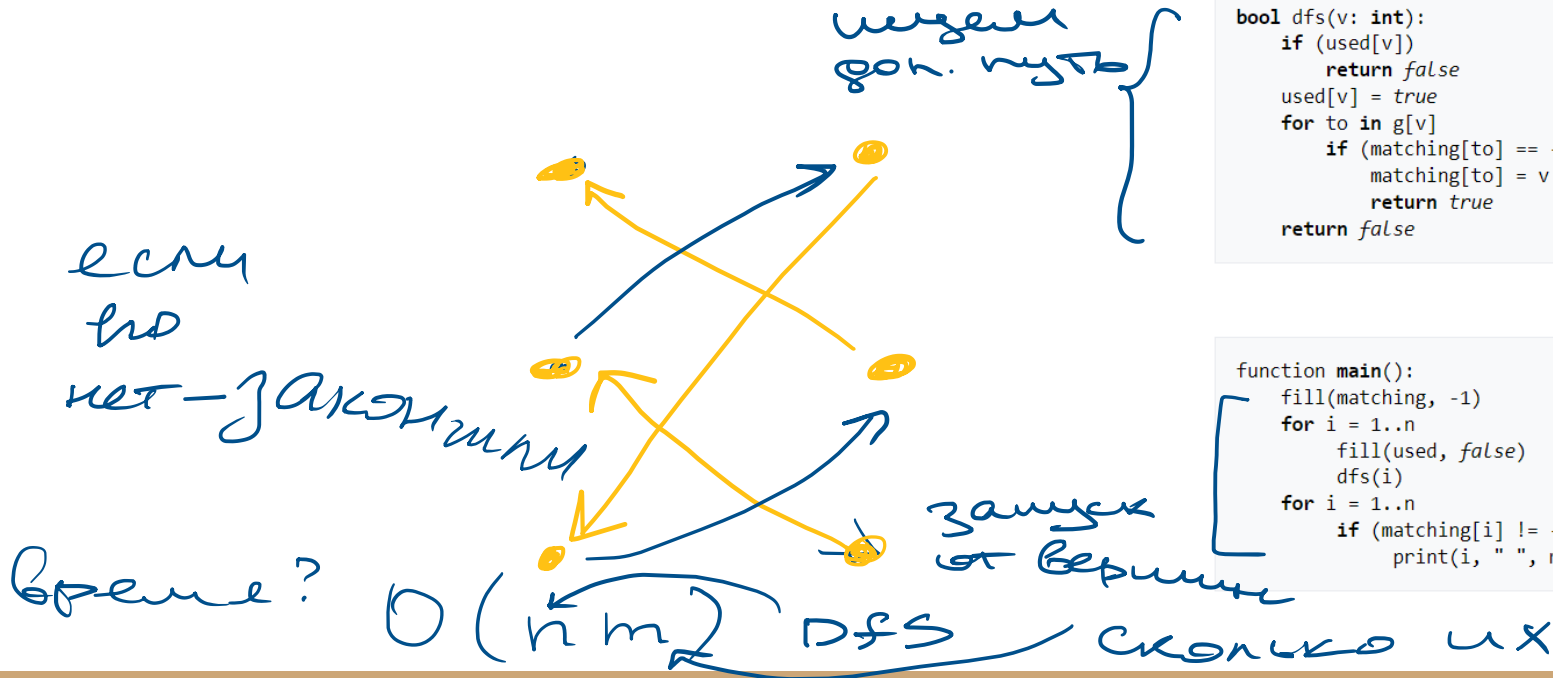
```
function main():  
    fill(matching, -1)  
    for i = 1..n  
        fill(used, false)  
        dfs(i)  
    for i = 1..n  
        if (matching[i] != -1)  
            print(i, " ", matching[i])
```

Задача о максимальном паросочетании.

Двудольный граф

Теорема:

Паросочетание M в двудольном графе G является максимальным тогда и только тогда, когда в G нет дополняющей цепи.



```
bool dfs(v: int):  
    if (used[v])  
        return false  
    used[v] = true  
    for to in g[v]  
        if (matching[to] == -1 or dfs(matching[to])):  
            matching[to] = v  
            return true  
    return false
```

```
function main():  
    fill(matching, -1)  
    for i = 1..n  
        fill(used, false)  
        dfs(i)  
    for i = 1..n  
        if (matching[i] != -1)  
            print(i, " ", matching[i])
```

Задача о максимальном паросочетании.

Двудольный граф

Теорема:

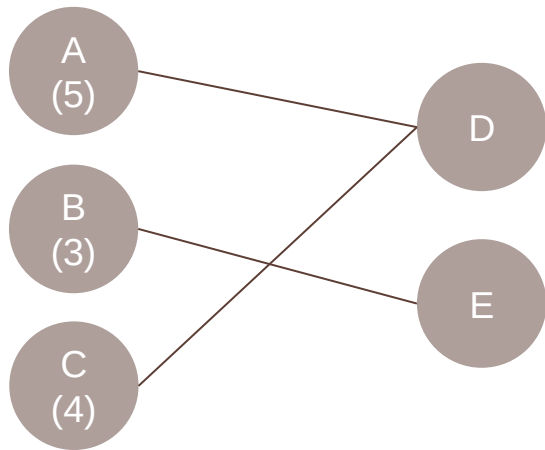
Паросочетание M в двудольном графе G является максимальным тогда и только тогда, когда в G нет дополняющей цепи.

Итого:

1. Начинаем с того, что все ребра свободны (развернуты вправо)
2. Выбираем путь слева направо
3. По пути обратно разворачиваем все ребра
4. Делаем так до тех пор, пока есть свободная вершина в левой доле
5. Успех!

Взвешенное максимальное паросочетание в двудольном графе.

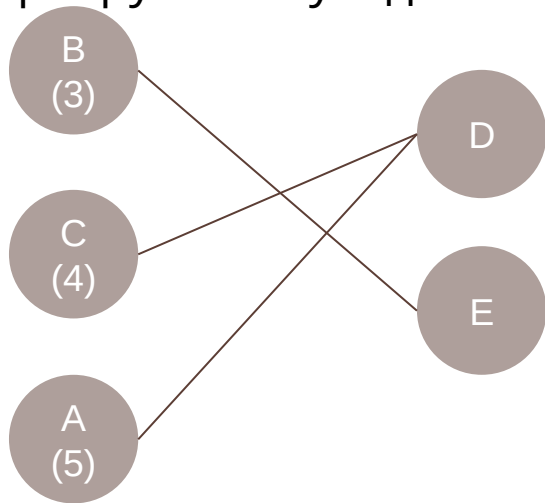
Пусть дан двудольный граф, где взвешена левая доля. Как найти макс паросочетание с максимальным весом входящих вершин?



Взвешенное максимальное паросочетание в двудольном графе.

Пусть дан двудольный граф, где взвешена левая доля. Как найти макс паросочетание с максимальным весом входящих вершин?

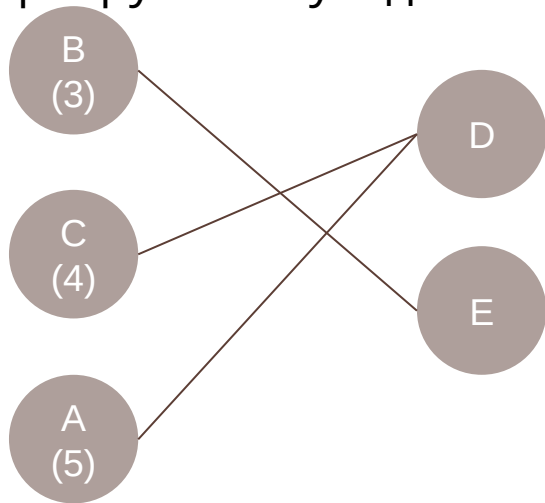
Давайте отсортируем левую долю



Взвешенное максимальное паросочетание в двудольном графе.

Пусть дан двудольный граф, где взвешена левая доля. Как найти макс паросочетание с максимальным весом входящих вершин?

Давайте отсортируем левую долю

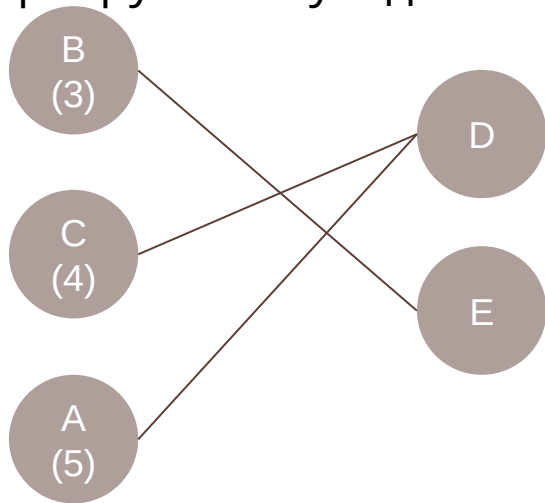


Теперь найдем максимальное паросочетание, стартуя с наиболее тяжелых вершин

Взвешенное максимальное паросочетание в двудольном графе.

Пусть дан двудольный граф, где взвешена левая доля. Как найти макс паросочетание с максимальным весом входящих вершин?

Давайте отсортируем левую долю



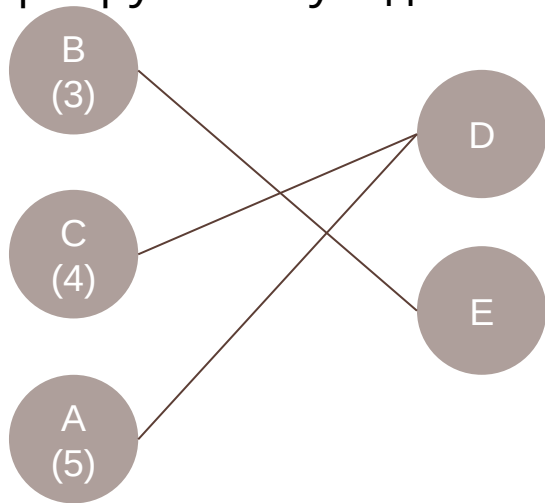
Теперь найдем максимальное паросочетание, стартуя с наиболее тяжелых вершин

Действуем жадно => найдем наиболее выгодное паросочетание

Взвешенное максимальное паросочетание в двудольном графе.

Пусть дан двудольный граф, где взвешена левая доля. Как найти макс паросочетание с максимальным весом входящих вершин?

Давайте отсортируем левую долю



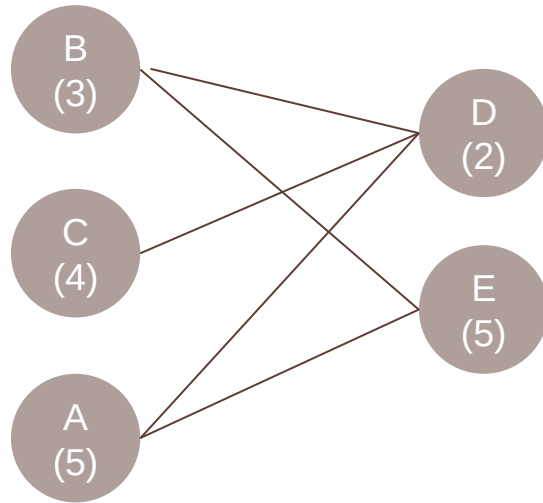
Теперь найдем максимальное паросочетание, стартуя с наиболее тяжелых вершин

Действуем жадно => найдем наиболее выгодное паросочетание

Асимптотика – сортировка + поиск паросочетания

Взвешенное максимальное паросочетание в двудольном графе.

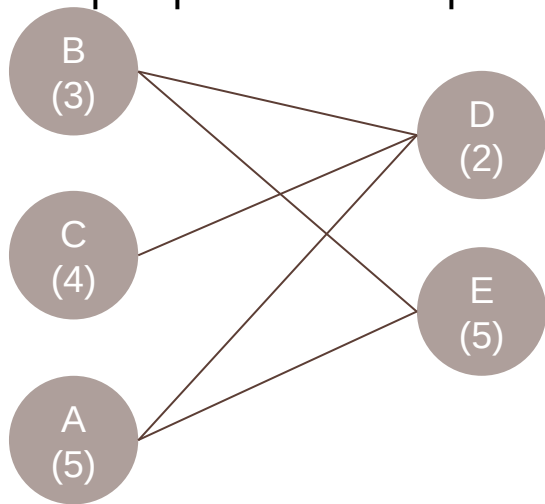
Пусть теперь взвешены обе доли.



Взвешенное максимальное паросочетание в двудольном графе.

Пусть теперь взвешены обе доли.

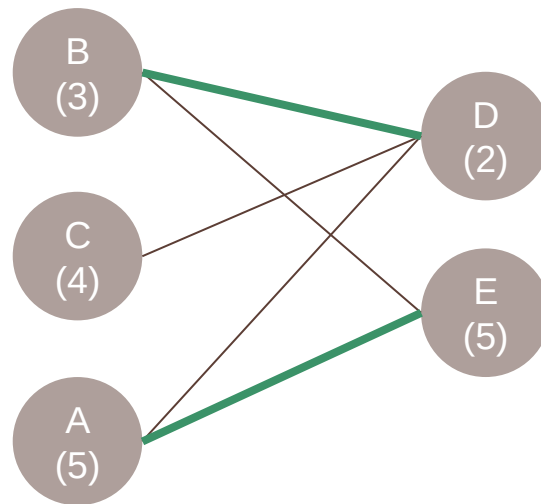
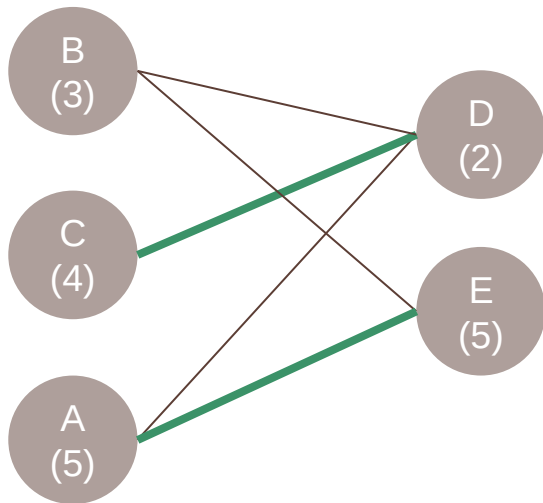
Найдем два паросочетания – одно по отсортированной левой доле, другое – по отсортированной правой доле.



Взвешенное максимальное паросочетание в двудольном графе.

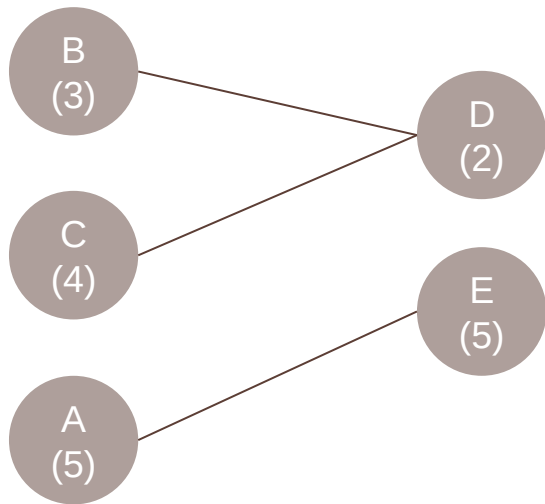
Пусть теперь взвешены обе доли.

Найдем два паросочетания – одно по отсортированной левой доле, другое – по отсортированной правой доле.



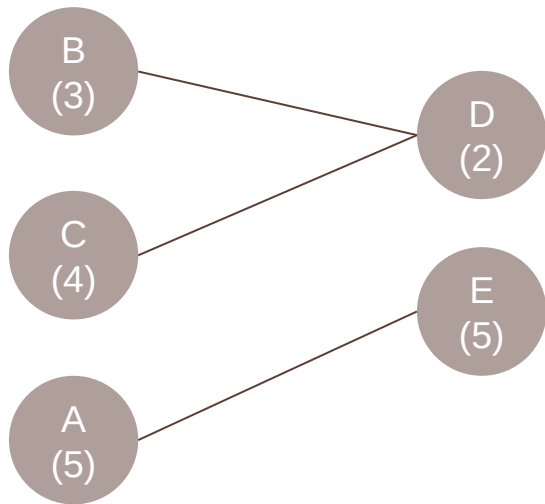
Взвешенное максимальное паросочетание в двудольном графе.

Оставим теперь только ребра, которые входят хотя бы в одно из найденных паросочетаний.



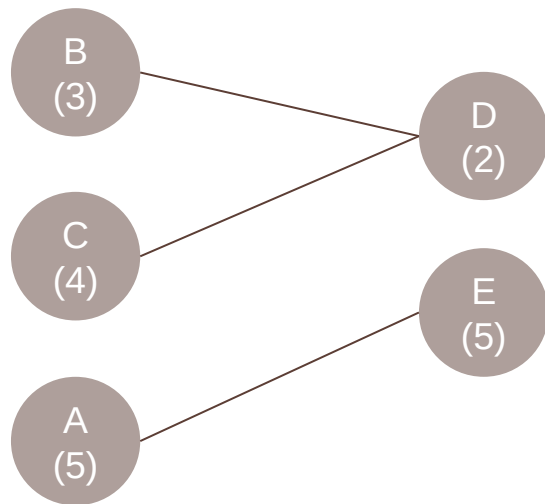
Взвешенное максимальное паросочетание в двудольном графе.

Найдем в этом графе максимальное паросочетание. Это и будет искомое паросочетание.



Взвешенное максимальное паросочетание в двудольном графе.

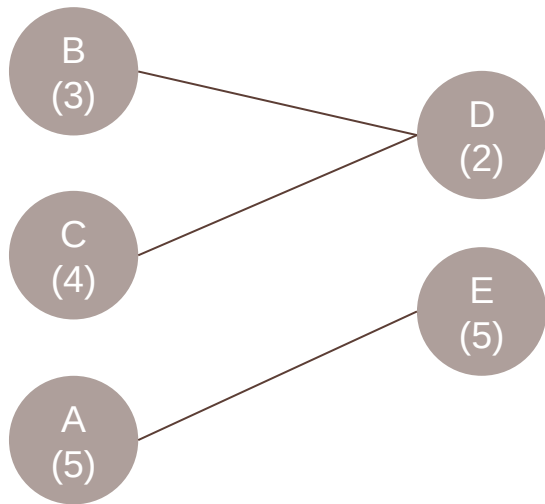
Найдем в этом графе максимальное паросочетание. Это и будет искомое паросочетание.



Почему работает?

Взвешенное максимальное паросочетание в двудольном графе.

Найдем в этом графе максимальное паросочетание. Это и будет искомое паросочетание.



Почему работает?

Мы нашли набор ребер, которые инцидентны вершинам с максимальным весом.

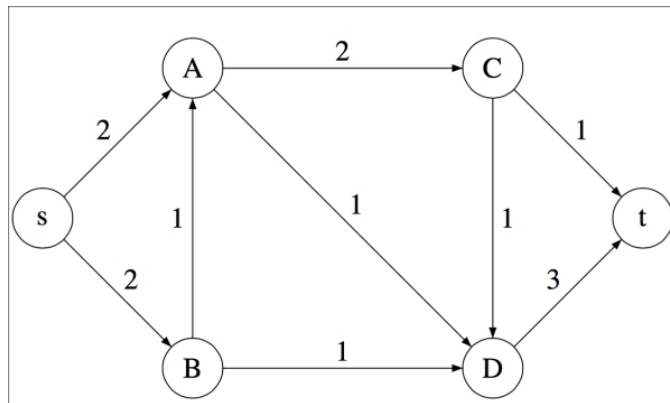
На этом наборе ребер ищем максимальное паросочетание
=> найдем максимальное паросочетание с максимальным весом

Задача о максимальном потоке. Сеть

Определение:

Сеть (англ. *flow network*) $G = (V, E)$ представляет собой **ориентированный граф**, в котором каждое **ребро** $(u, v) \in E$ имеет положительную **пропускную способность** (англ. *capacity*) $c(u, v) > 0$. Если $(u, v) \notin E$, предполагается что $c(u, v) = 0$.

В транспортной сети выделяются две вершины: **исток** s и **сток** t .

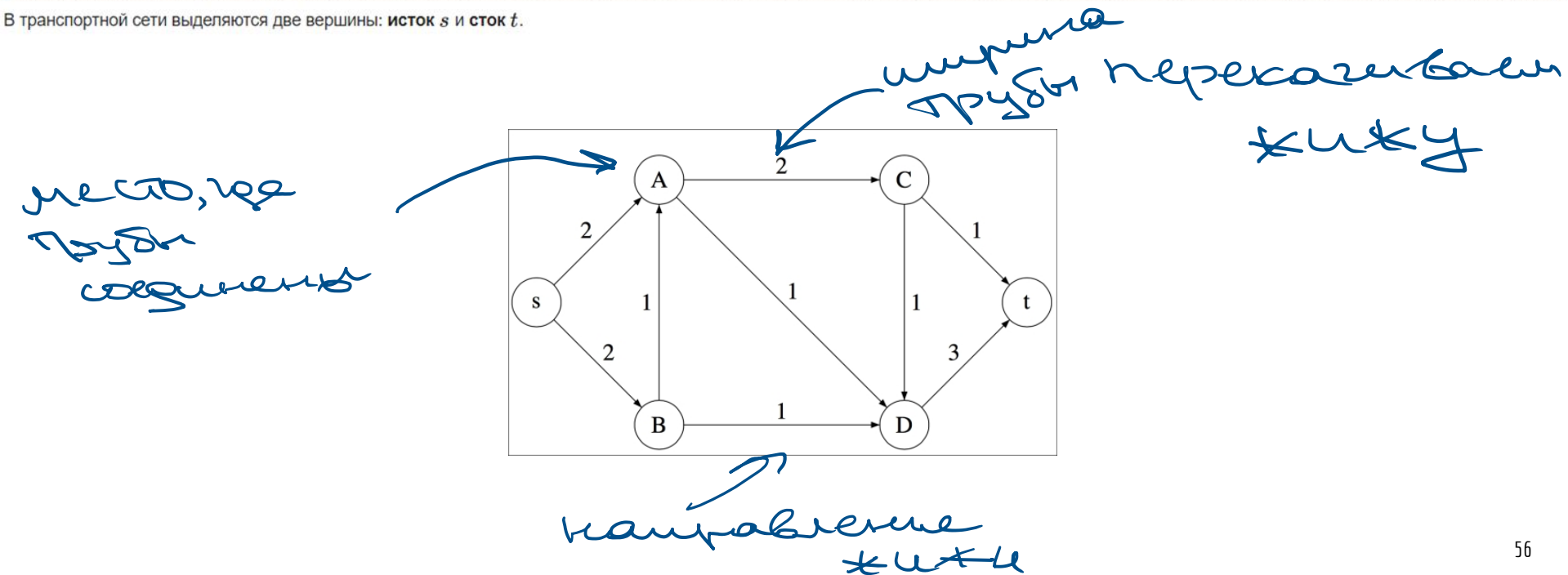


Задача о максимальном потоке. Сеть

Определение:

Сеть (англ. *flow network*) $G = (V, E)$ представляет собой **ориентированный граф**, в котором каждое **ребро** $(u, v) \in E$ имеет положительную **пропускную способность** (англ. *capacity*) $c(u, v) > 0$. Если $(u, v) \notin E$, предполагается что $c(u, v) = 0$.

В транспортной сети выделяются две вершины: **исток** s и **сток** t .



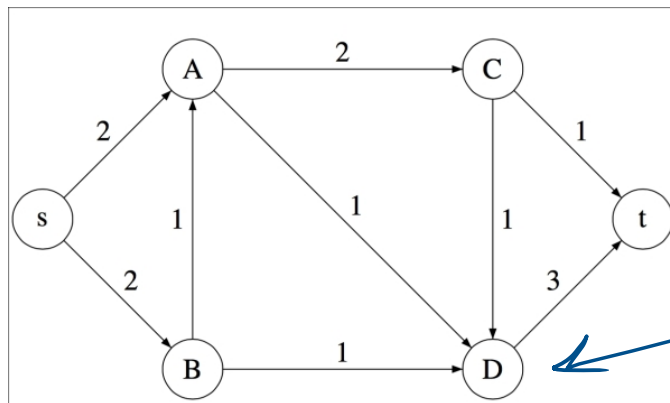
Задача о максимальном потоке. Поток

Определение:

Потоком (англ. *flow*) f в G является действительная функция $f: V \times V \rightarrow R$, удовлетворяющая условиям:

- 1) $f(u, v) = -f(v, u)$ (антисимметричность);
- 2) $f(u, v) \leq c(u, v)$ (ограничение пропускной способности), если ребра нет, то $f(u, v) = 0$;
- 3) $\sum_v f(u, v) = 0$ для всех вершин u , кроме s и t (закон сохранения потока).

Величина потока f определяется как $|f| = \sum_{v \in V} f(s, v)$.



используем
хитрости
теперь
через
граф

совмещено
в узлах

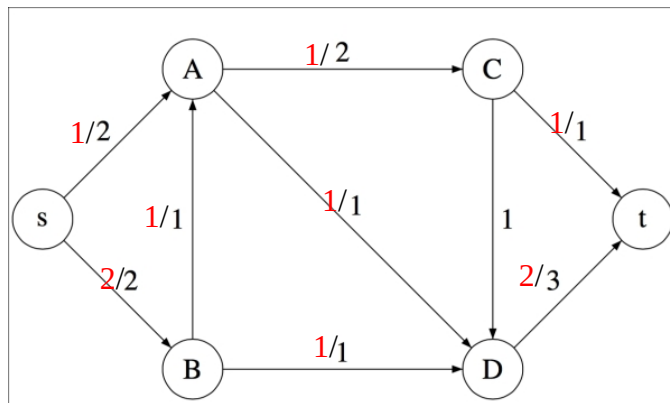
Задача о максимальном потоке. Пример потока

Определение:

Потоком (англ. *flow*) f в G является действительная функция $f: V \times V \rightarrow R$, удовлетворяющая условиям:

- 1) $f(u, v) = -f(v, u)$ (антисимметричность);
- 2) $f(u, v) \leq c(u, v)$ (ограничение пропускной способности), если ребра нет, то $f(u, v) = 0$;
- 3) $\sum_v f(u, v) = 0$ для всех вершин u , кроме s и t (закон сохранения потока).

Величина потока f определяется как $|f| = \sum_{v \in V} f(s, v)$.



Задача о максимальном потоке. Остаточная сеть

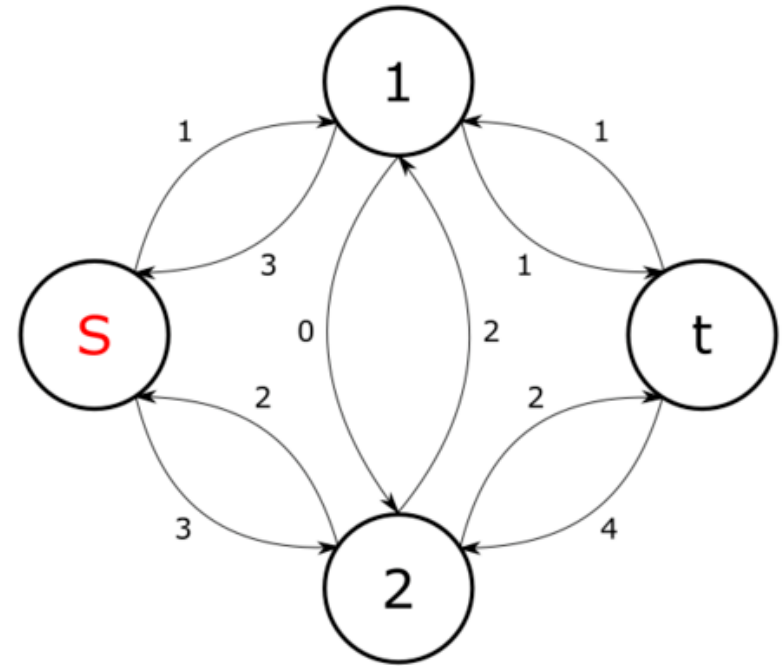
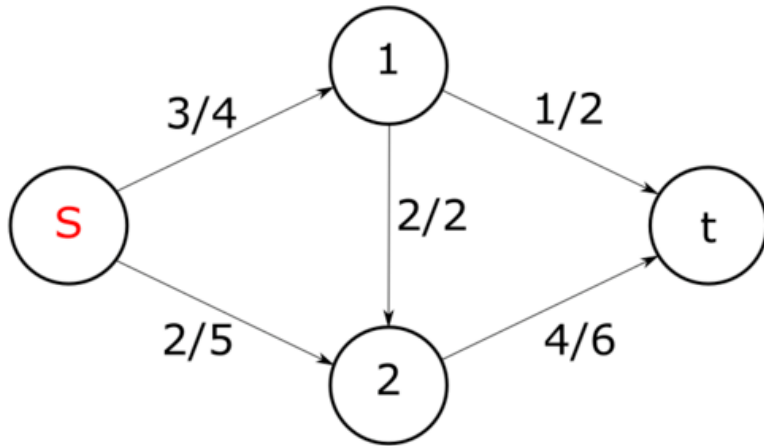
Определение:

Остаточной пропускной способностью (англ. *residual capacity*) ребра (u, v) называется величина дополнительного **потока**, который мы можем направить из u в v , не превысив **пропускную способность** $c(u, v)$. Иными словами $c_f(u, v) = c(u, v) - f(u, v)$.

Определение:

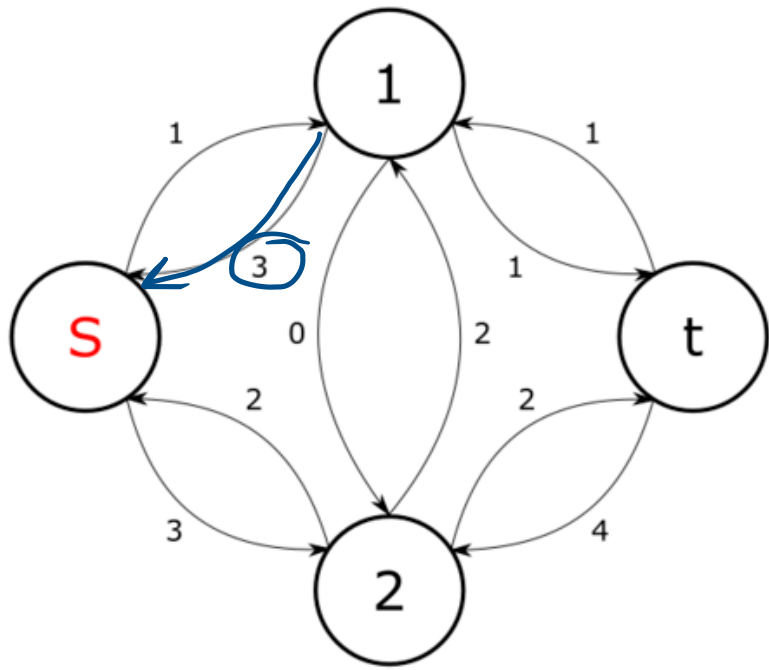
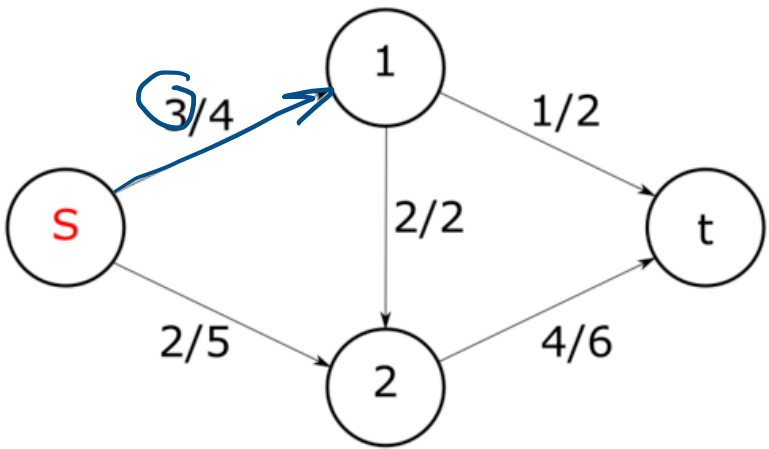
Для заданной **транспортной сети** $G = (V, E)$ и потока f , **остаточной сетью**, (**дополняющая сеть**, англ. *residual network*) в G , порожденной потоком f , является сеть $G_f = (V, E_f)$, где $E_f = \{(u, v) \in V \times V \mid c_f(u, v) > 0\}$

Задача о максимальном потоке. Остаточная сеть



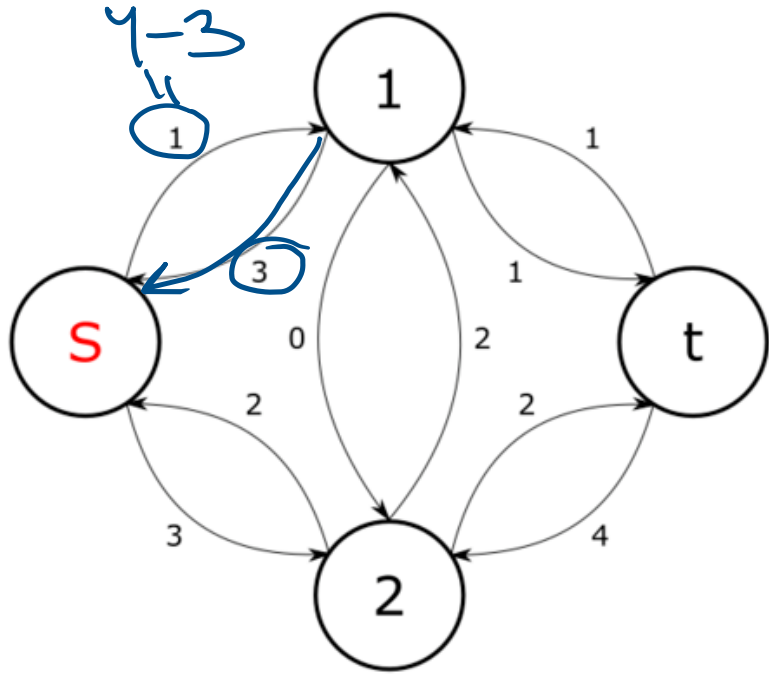
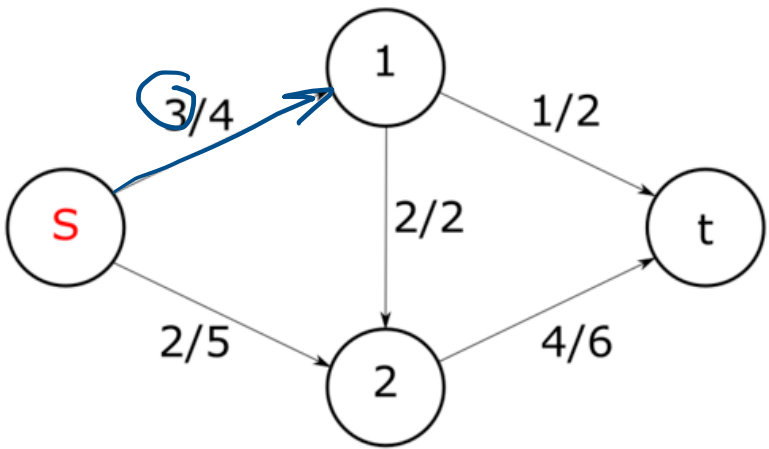
обратные ребра появляются из-за свойства антисимметричности (см. 46 сл.)

Задача о максимальном потоке. Остаточная сеть



обратные ребра появляются из-за свойства антисимметричности (см. 46 сл.)

Задача о максимальном потоке. Остаточная сеть

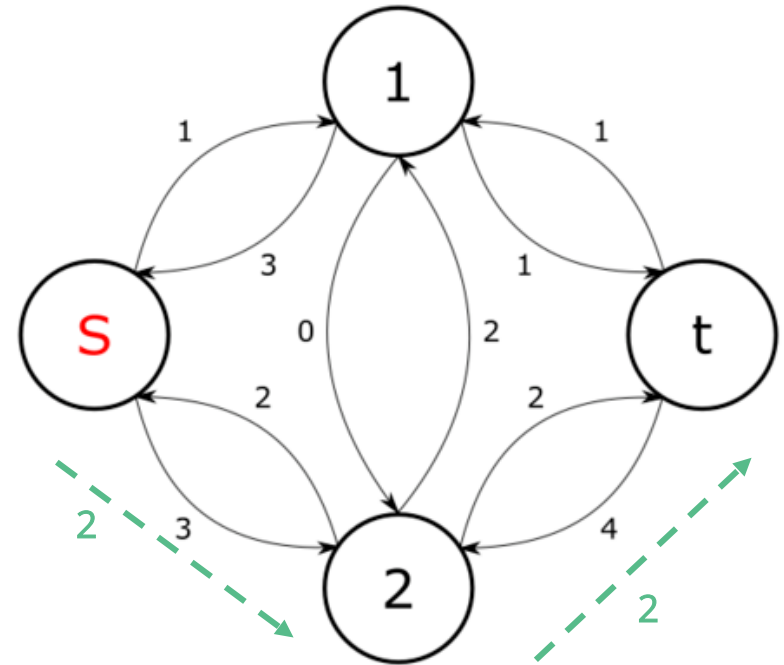
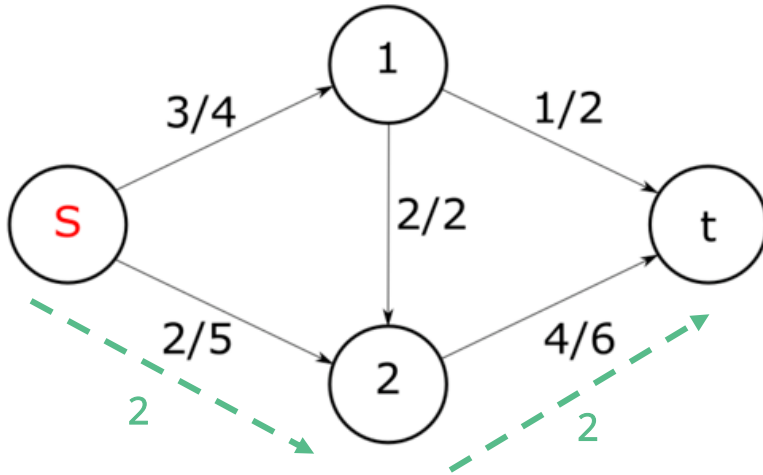


обратные ребра появляются из-за свойства антисимметричности (см. 46 сл.)

Задача о максимальном потоке. Дополняющий путь

Определение:

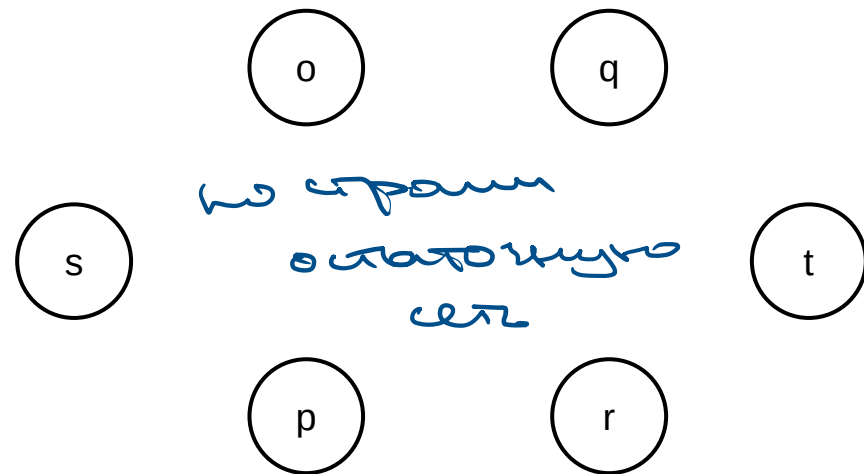
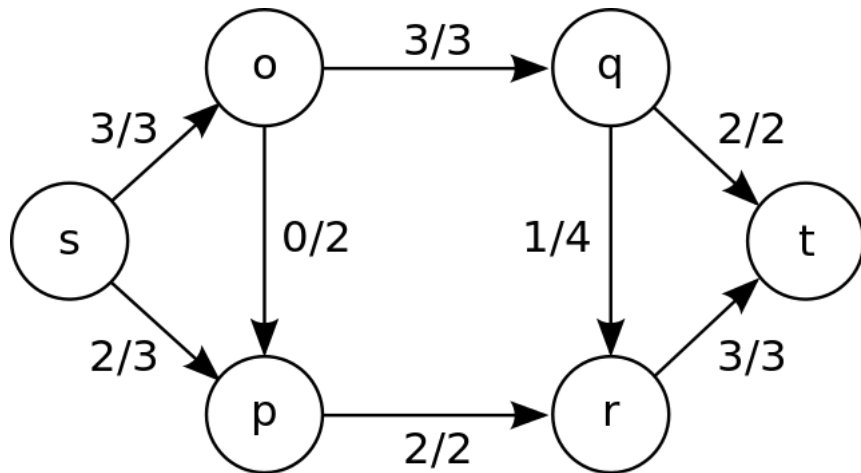
Для заданной транспортной сети $G = (V, E)$ и потока f дополняющим путем (англ. *augmenting path*) p является простой путь из **источка** в **сток** в остаточной сети $G_f = (V, E_f)$.



Теорема Форда-Фалкерсона

Если f — некоторый поток в сети $G = (V, E)$ с источником s и стоком t , то следующие утверждения эквивалентны:

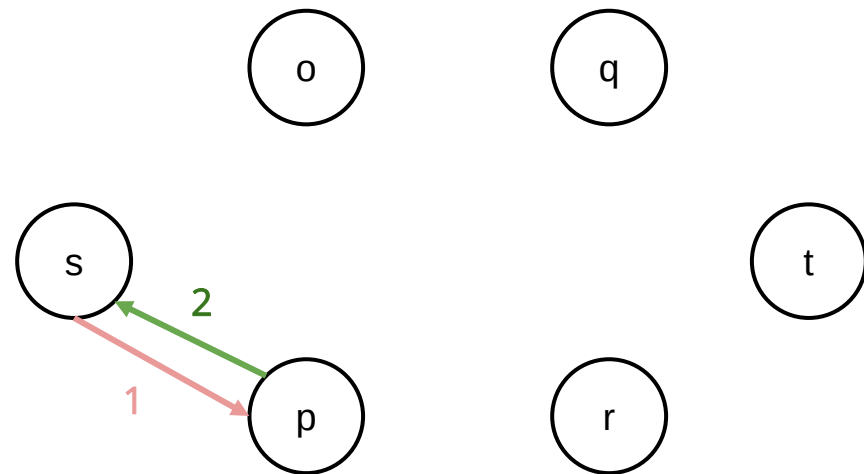
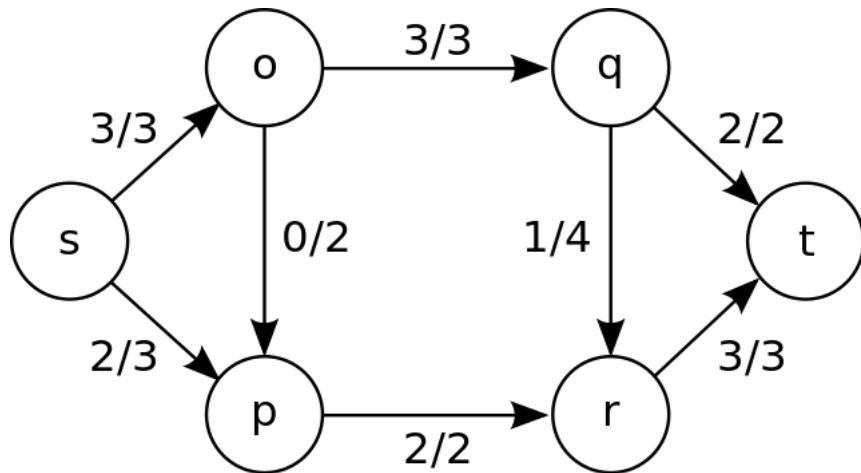
1. Поток f максимален.
2. В G_f не существует пути $s \rightsquigarrow t$.
3. $|f| = c(S, T)$ для некоторого разреза (S, T) сети G .



Теорема Форда-Фалкерсона

Если f — некоторый поток в сети $G = (V, E)$ с источником s и стоком t , то следующие утверждения эквивалентны:

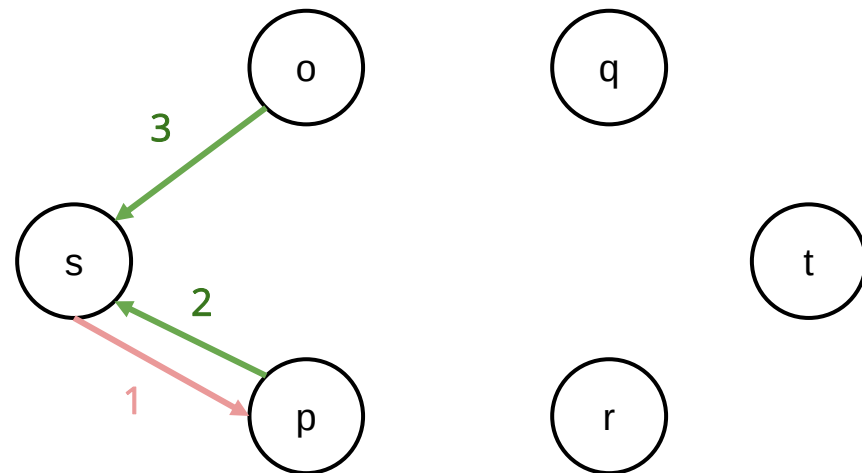
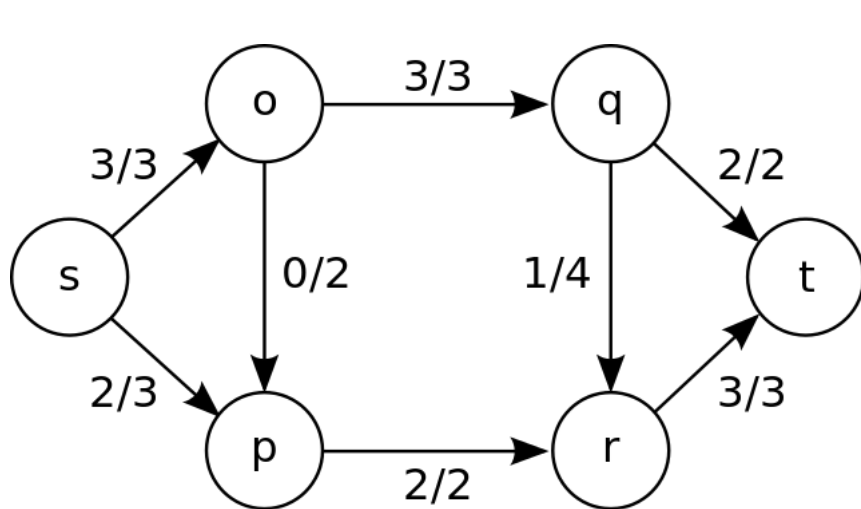
1. Поток f максимален.
2. В G_f не существует пути $s \rightsquigarrow t$.
3. $|f| = c(S, T)$ для некоторого разреза (S, T) сети G .



Теорема Форда-Фалкерсона

Если f — некоторый поток в сети $G = (V, E)$ с источником s и стоком t , то следующие утверждения эквивалентны:

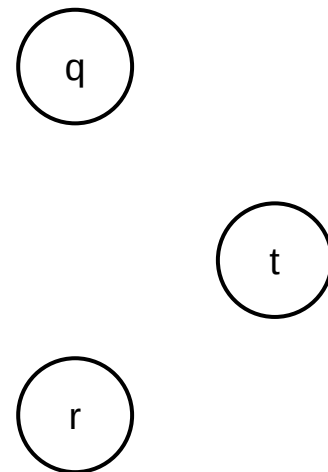
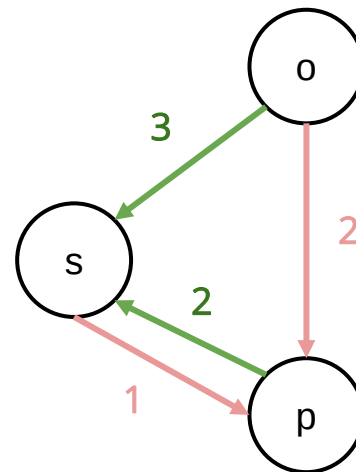
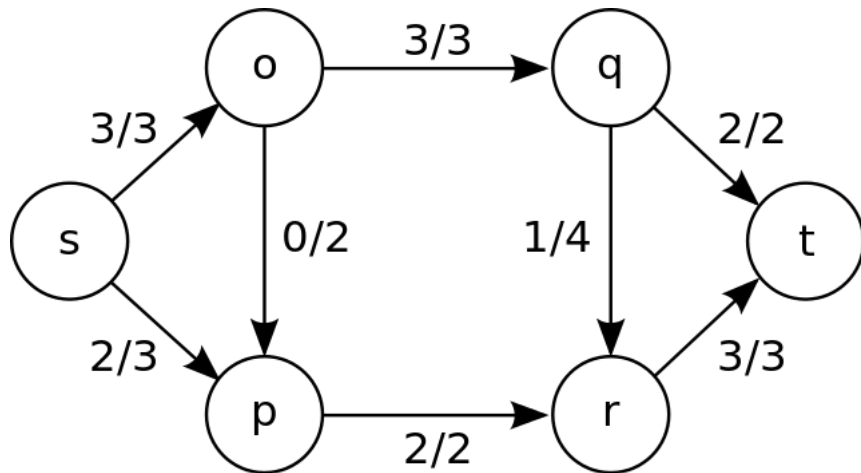
1. Поток f максимален.
2. В G_f не существует пути $s \rightsquigarrow t$.
3. $|f| = c(S, T)$ для некоторого разреза (S, T) сети G .



Теорема Форда-Фалкерсона

Если f — некоторый поток в сети $G = (V, E)$ с источником s и стоком t , то следующие утверждения эквивалентны:

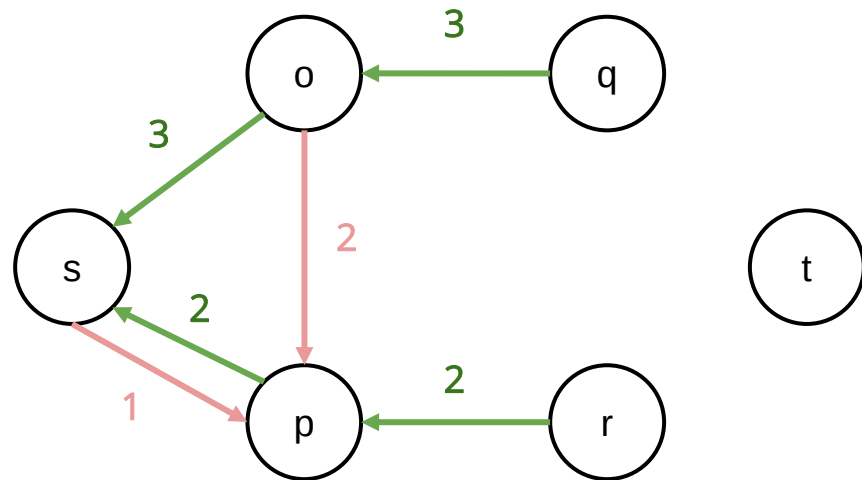
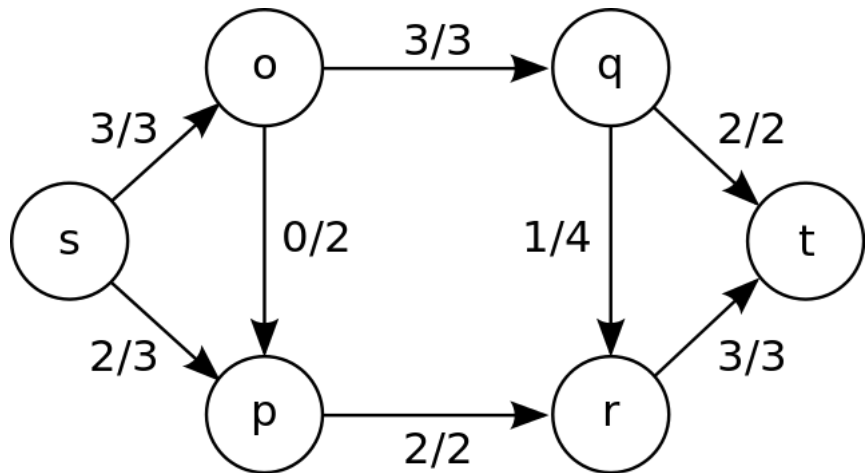
1. Поток f максимален.
2. В G_f не существует пути $s \rightsquigarrow t$.
3. $|f| = c(S, T)$ для некоторого разреза (S, T) сети G .



Теорема Форда-Фалкерсона

Если f — некоторый поток в сети $G = (V, E)$ с источником s и стоком t , то следующие утверждения эквивалентны:

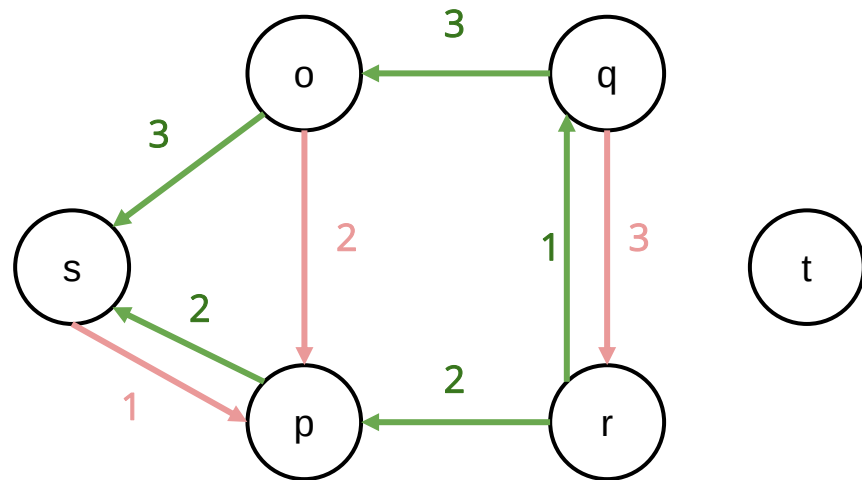
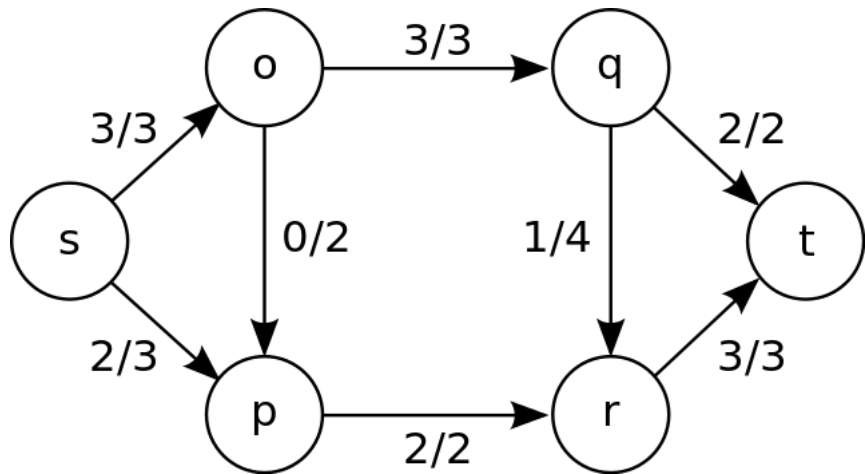
1. Поток f максимален.
2. В G_f не существует пути $s \rightsquigarrow t$.
3. $|f| = c(S, T)$ для некоторого разреза (S, T) сети G .



Теорема Форда-Фалкерсона

Если f — некоторый поток в сети $G = (V, E)$ с источником s и стоком t , то следующие утверждения эквивалентны:

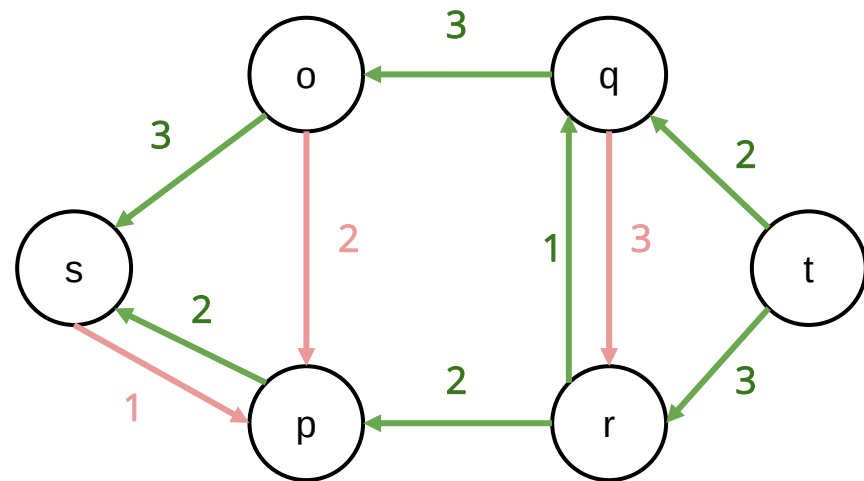
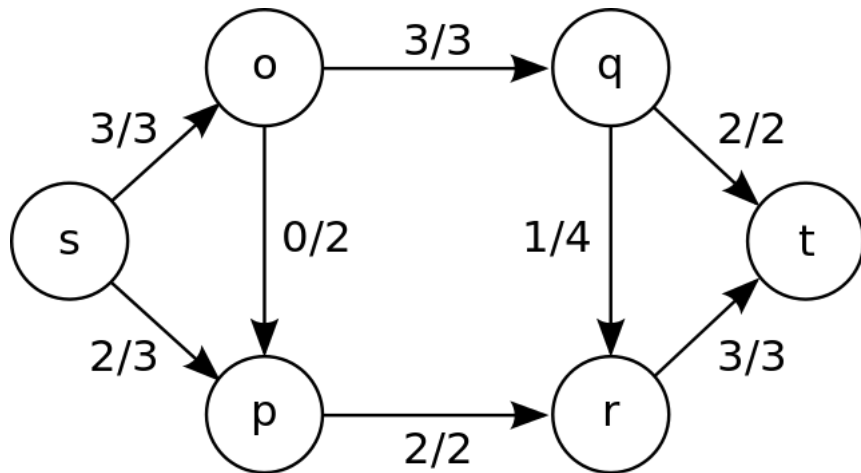
1. Поток f максимален.
2. В G_f не существует пути $s \rightsquigarrow t$.
3. $|f| = c(S, T)$ для некоторого разреза (S, T) сети G .



Теорема Форда-Фалкерсона

Если f — некоторый поток в сети $G = (V, E)$ с источником s и стоком t , то следующие утверждения эквивалентны:

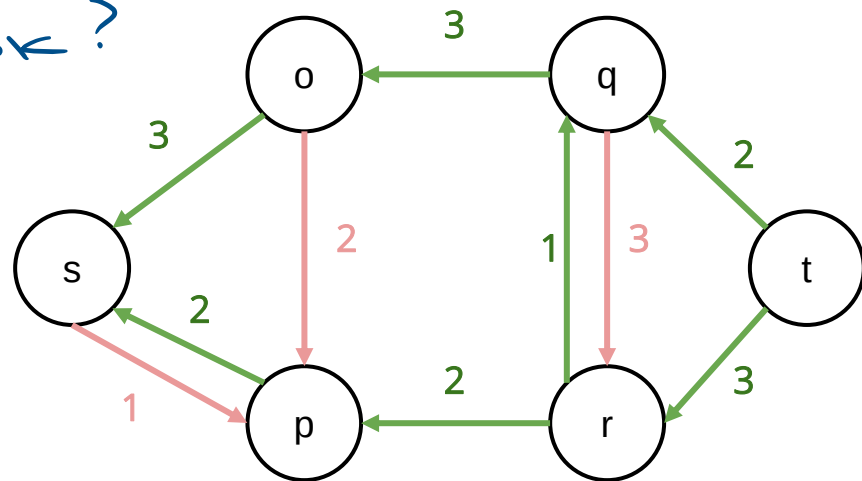
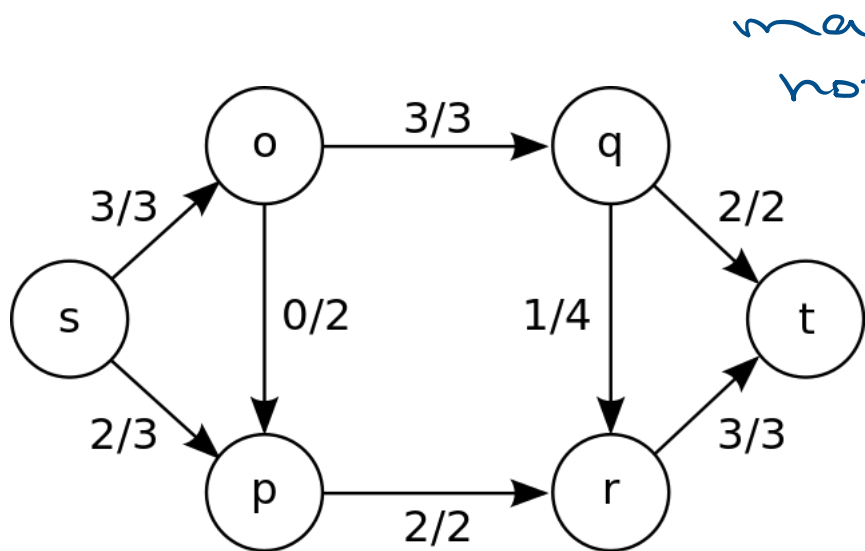
1. Поток f максимален.
2. В G_f не существует пути $s \rightsquigarrow t$.
3. $|f| = c(S, T)$ для некоторого разреза (S, T) сети G .



Теорема Форда-Фалкерсона

Если f — некоторый поток в сети $G = (V, E)$ с источником s и стоком t , то следующие утверждения эквивалентны:

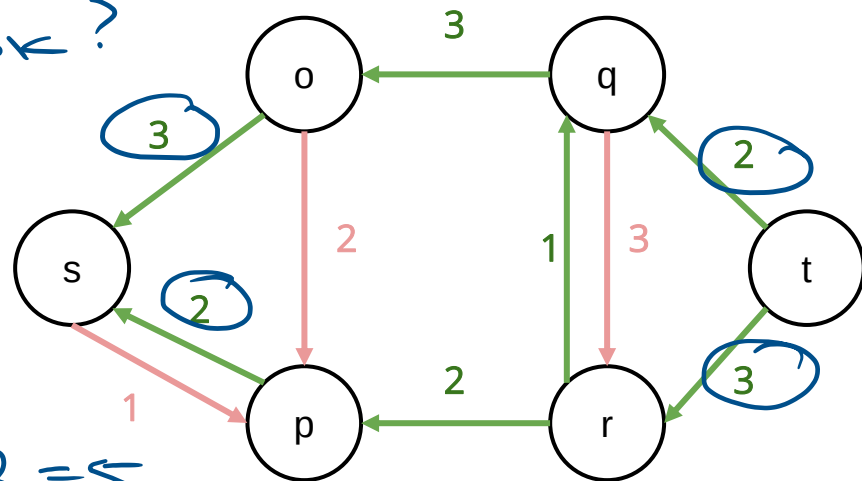
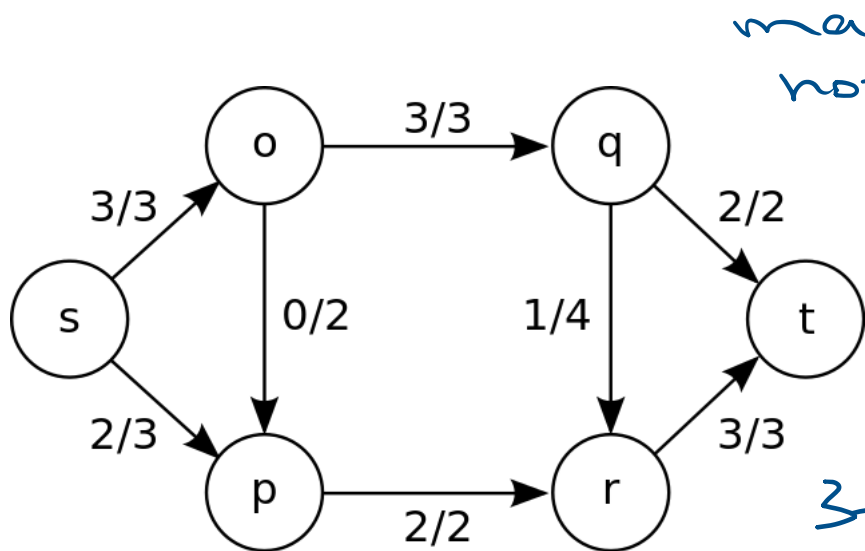
1. Поток f максимален.
2. В G_f не существует пути $s \rightsquigarrow t$.
3. $|f| = c(S, T)$ для некоторого разреза (S, T) сети G .



Теорема Форда-Фалкерсона

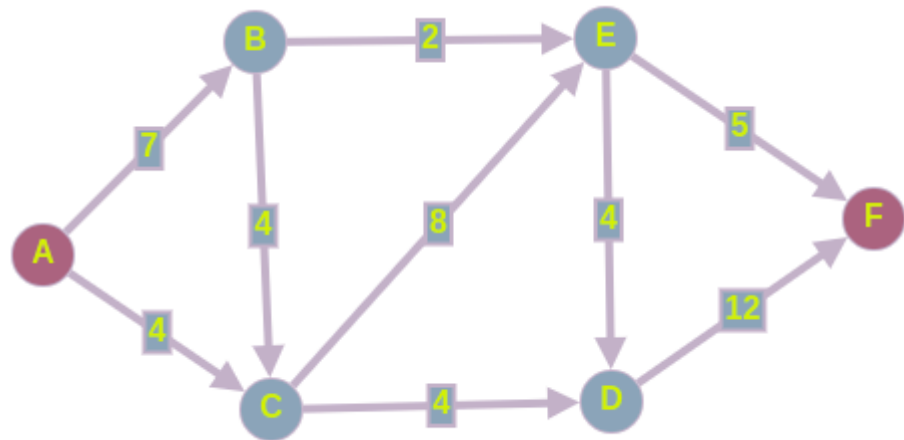
Если f — некоторый поток в сети $G = (V, E)$ с источником s и стоком t , то следующие утверждения эквивалентны:

1. Поток f максимален.
2. В G_f не существует пути $s \rightsquigarrow t$.
3. $|f| = c(S, T)$ для некоторого разреза (S, T) сети G .



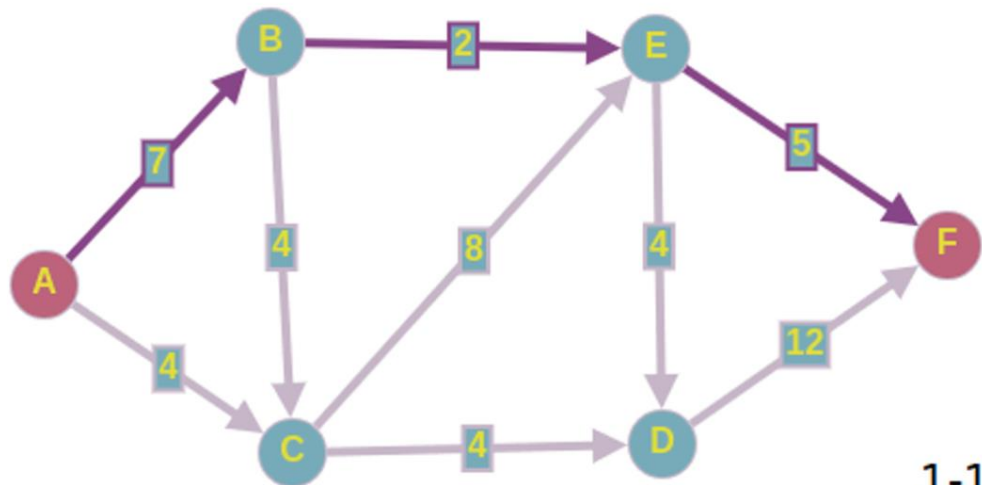
$$3 + 2 = 5$$

Алгоритм Форда-Фалкерсона

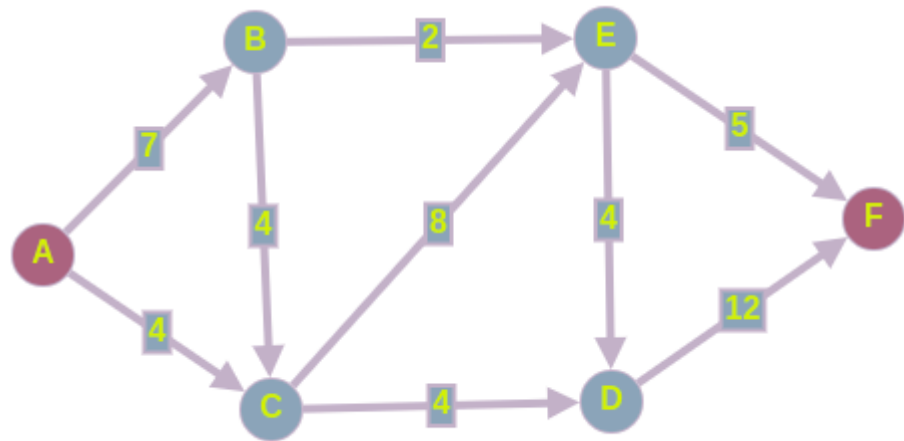


Исходный граф

Остаточная сеть



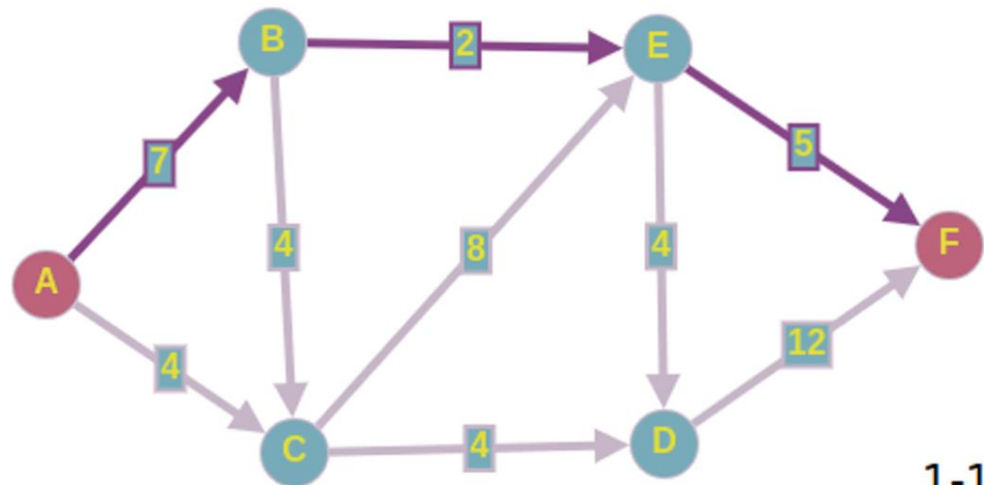
Алгоритм Форда-Фалкерсона



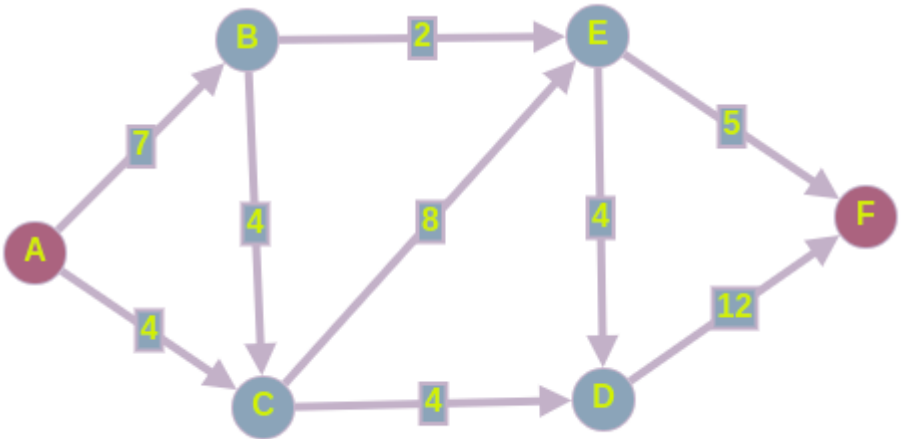
Исходный граф

$$w_{\text{max}} = 2$$

Остаточная сеть

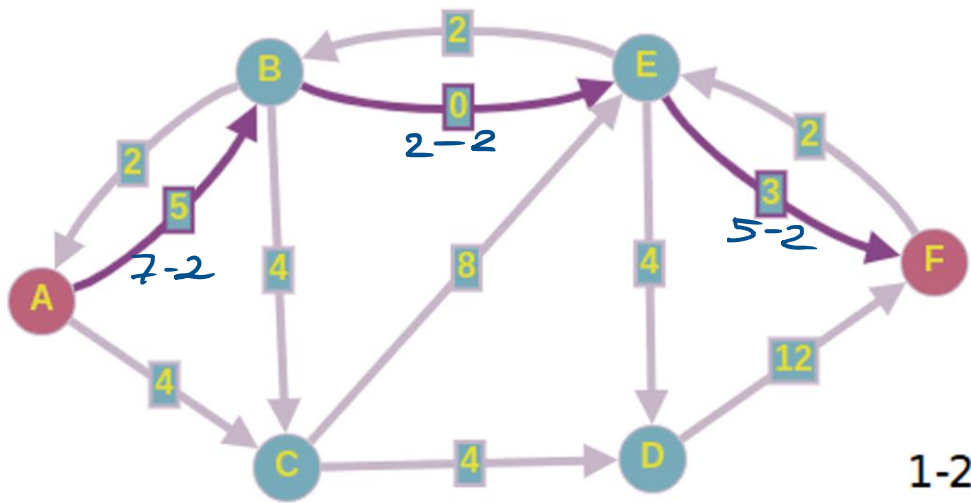


Алгоритм Форда-Фалкерсона

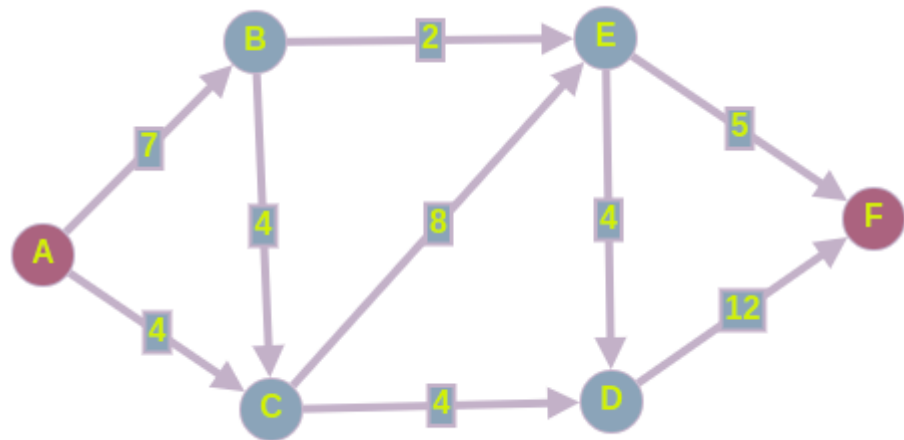


Исходный граф

Остаточная сеть



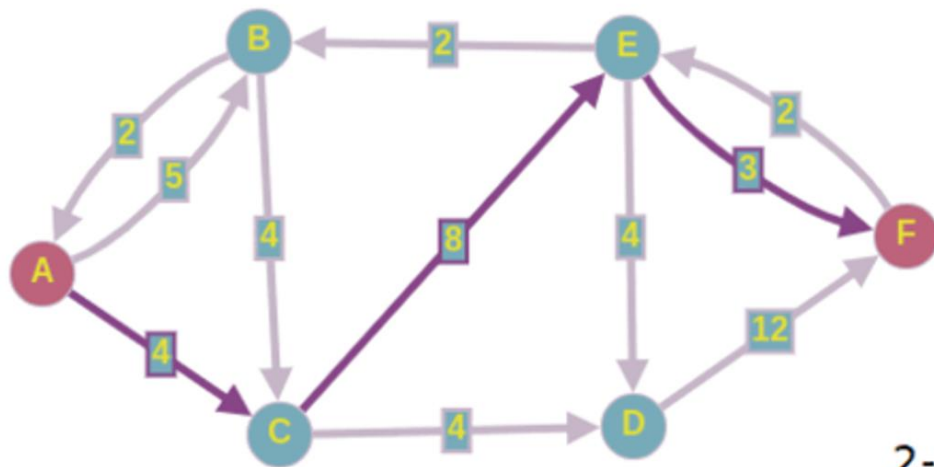
Алгоритм Форда-Фалкерсона



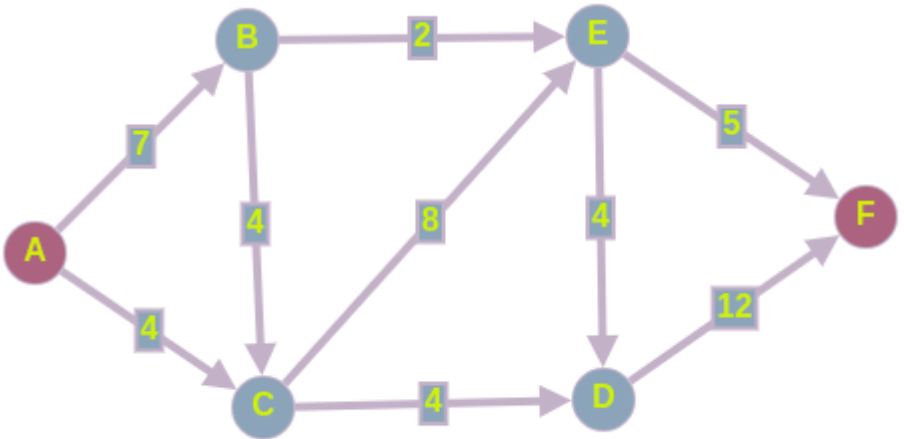
Исходный граф

$c_{min} - ?$

Остаточная сеть



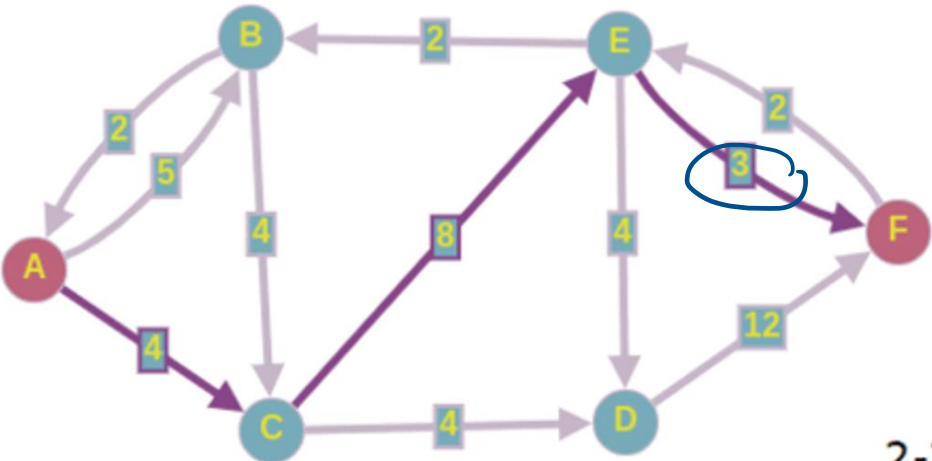
Алгоритм Форда-Фалкерсона



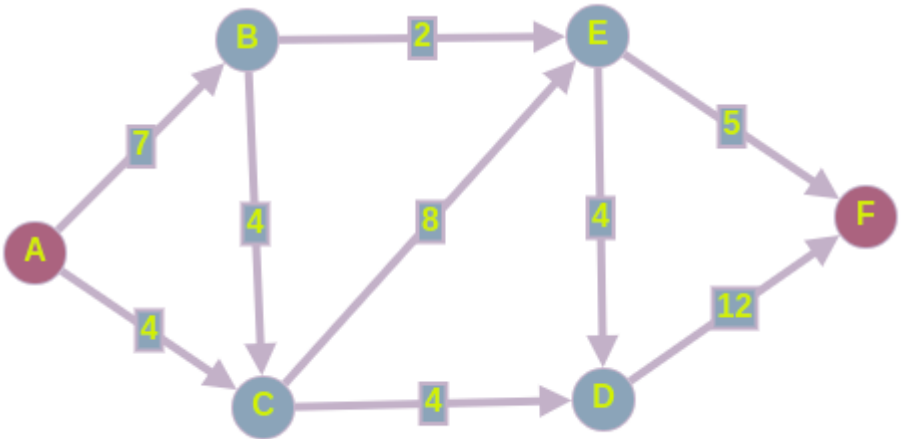
Исходный граф

$c_{min} = ?$
3

Остаточная сеть

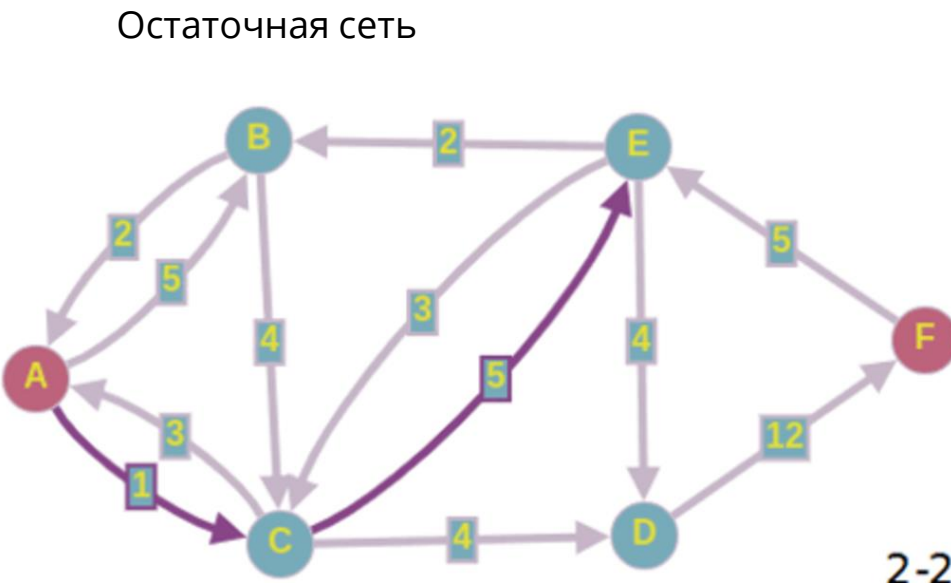


Алгоритм Форда-Фалкерсона

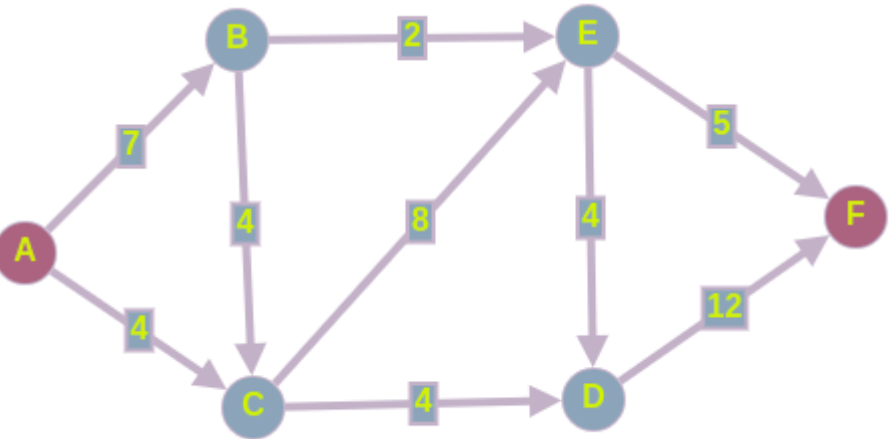


Исходный граф

вычитаем
на пути

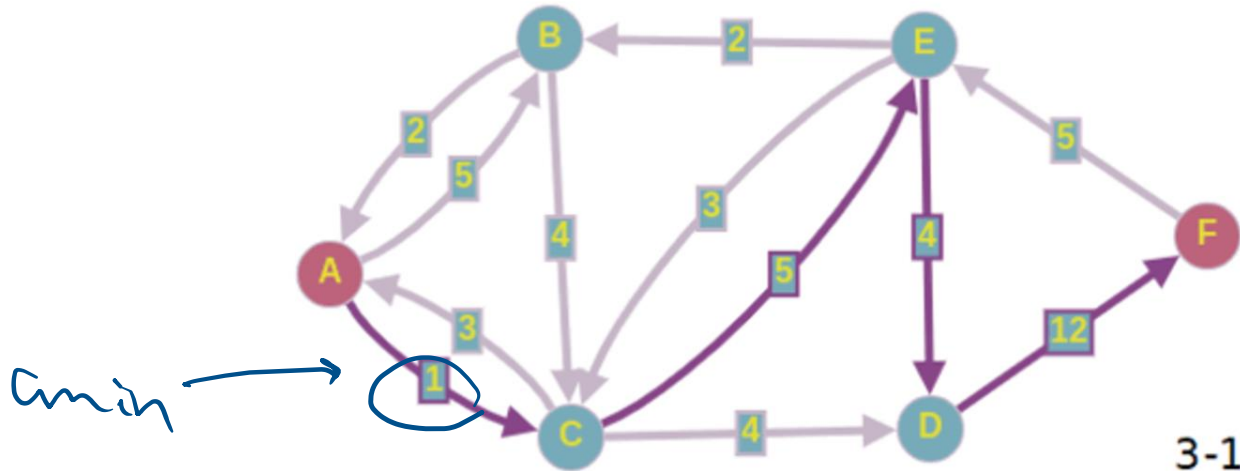


Алгоритм Форда-Фалкерсона

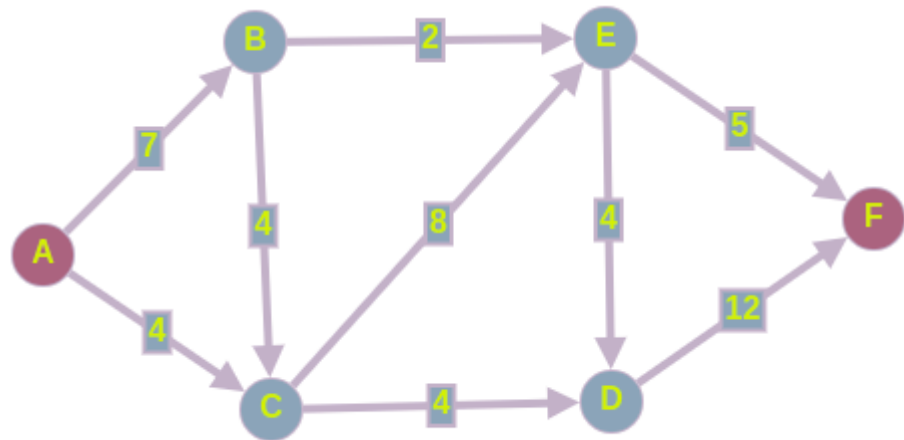


Исходный граф

Остаточная сеть

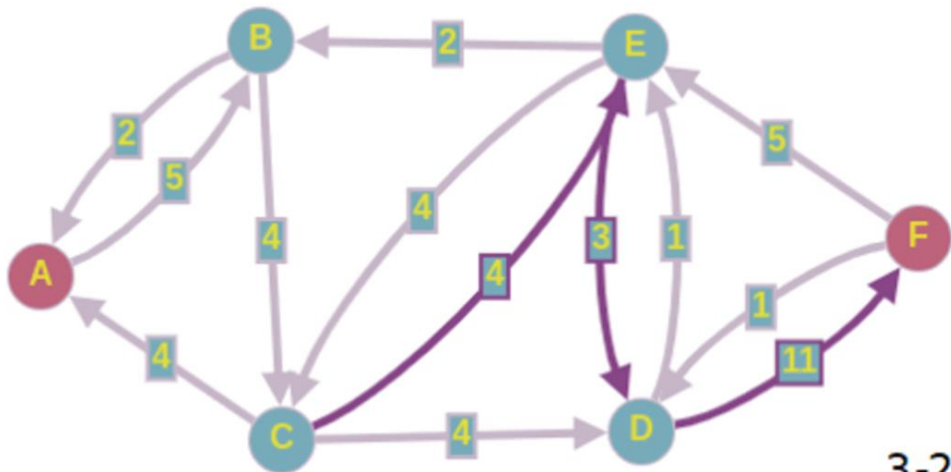


Алгоритм Форда-Фалкерсона

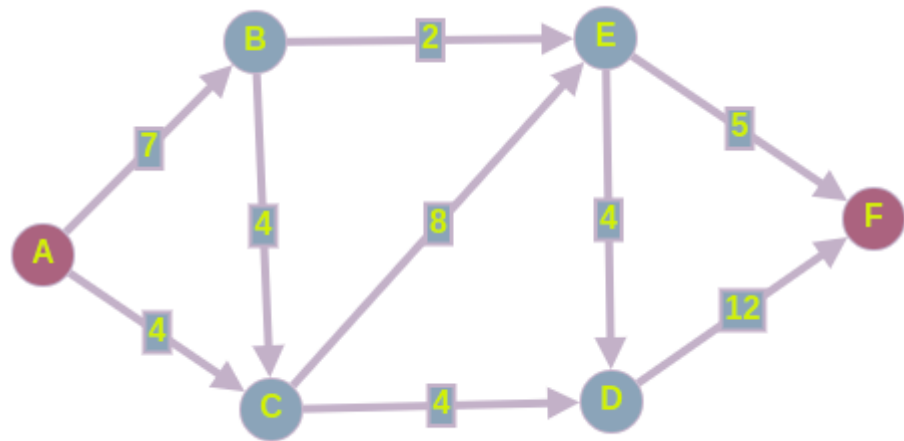


Исходный граф

Остаточная сеть

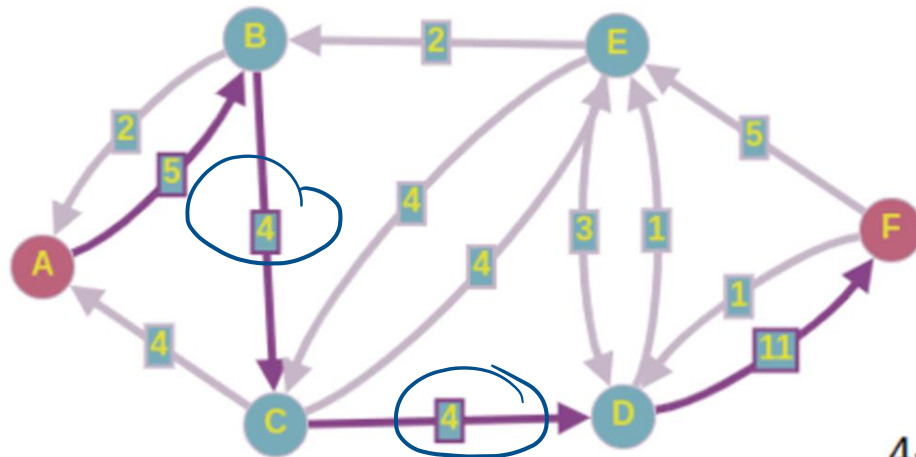


Алгоритм Форда-Фалкерсона

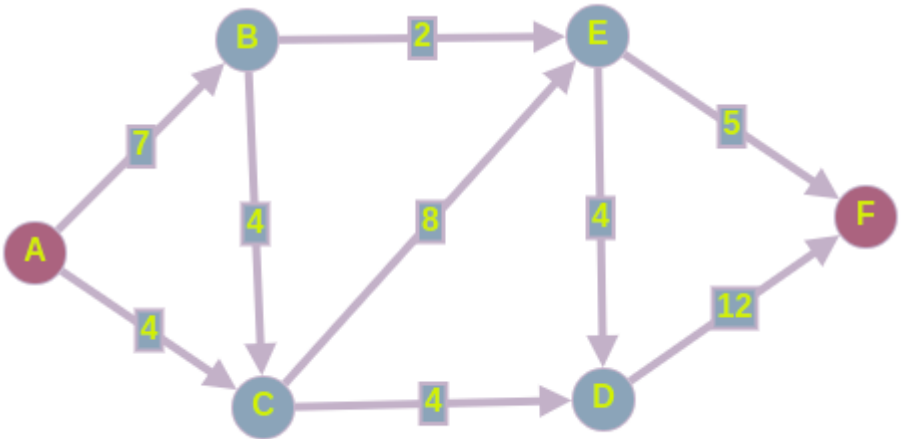


Исходный граф

Остаточная сеть

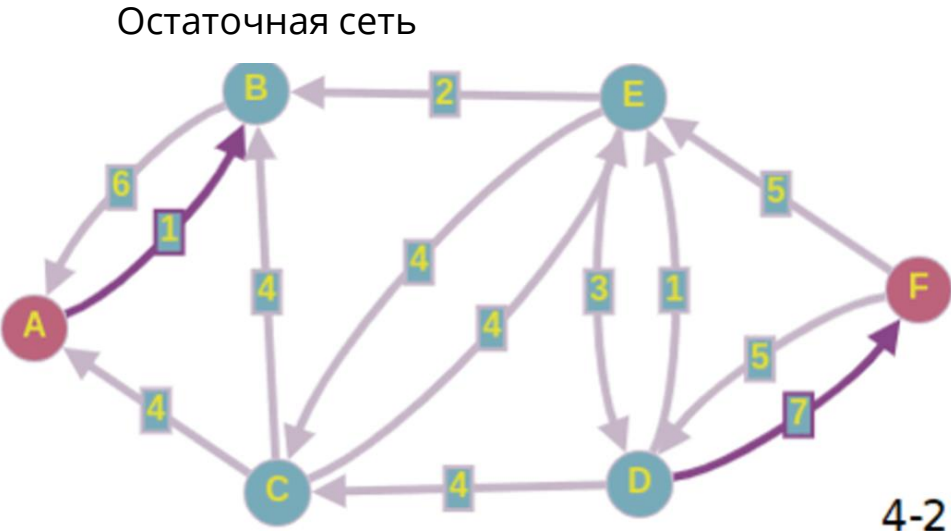


Алгоритм Форда-Фалкерсона



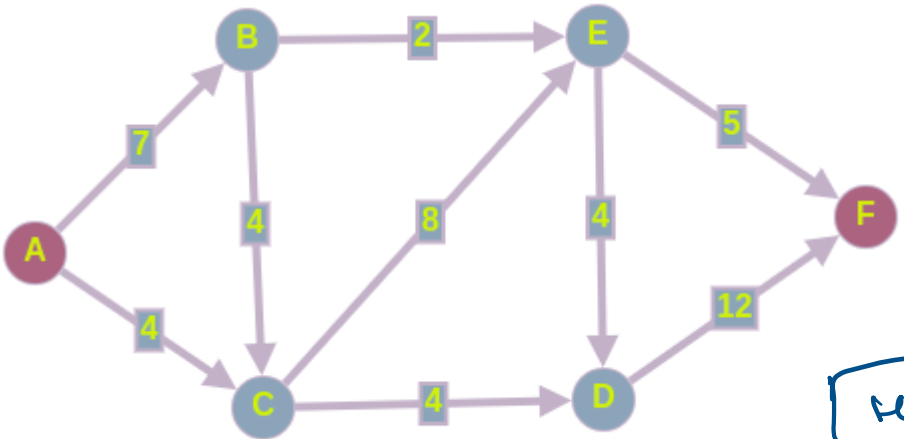
Исходный граф

Есть ли еще путь?



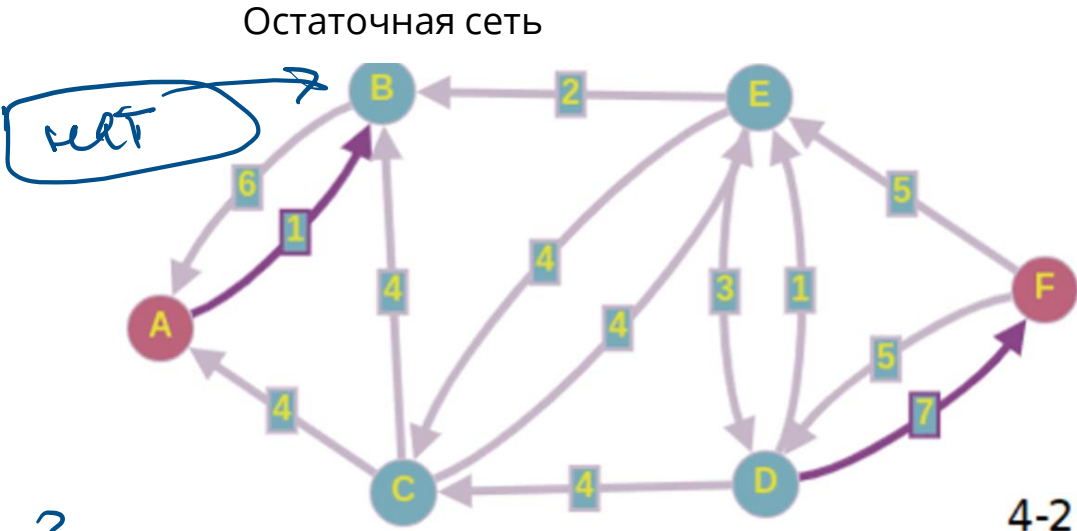
Остаточная сеть

Алгоритм Форда-Фалкерсона

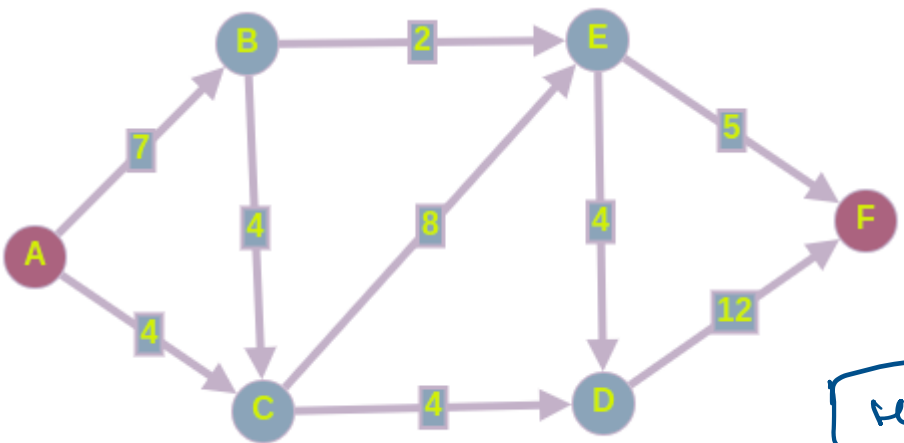


Исходный граф

Есть ли еще путь?



Алгоритм Форда-Фалкерсона



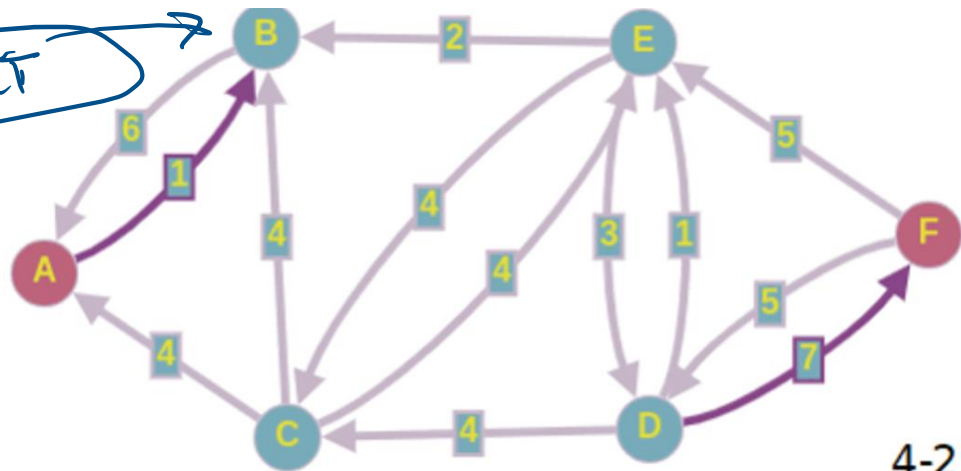
Исходный граф

поток через сеть?

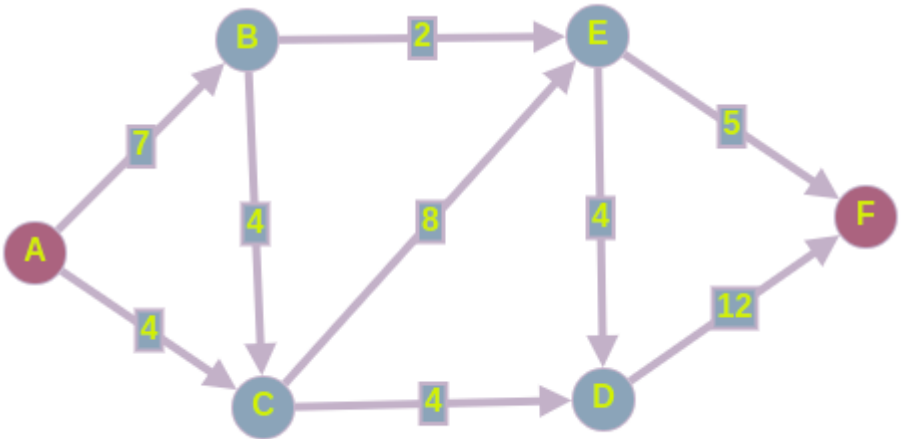
Есть ли еще путь?

нет

Остаточная сеть



Алгоритм Форда-Фалкерсона



Исходный граф

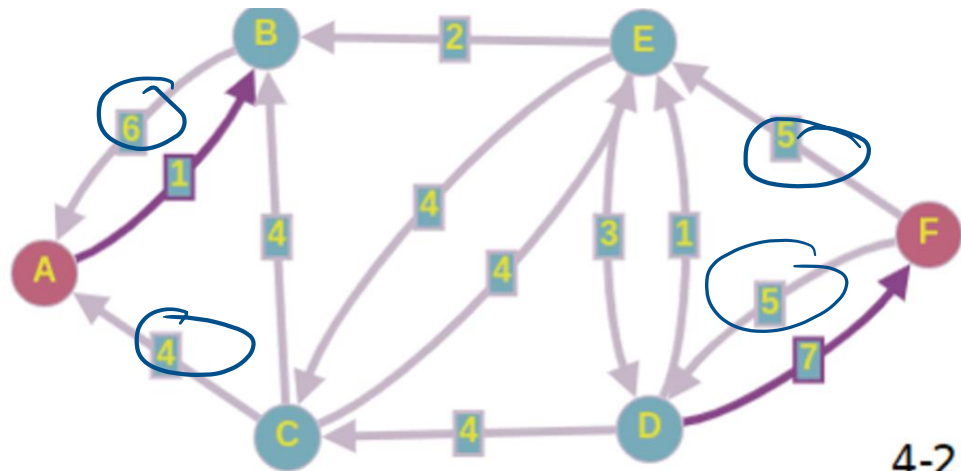
$6 + 4 = 10$

$5 + 5 = 10$

поток через сеть?

10

Остаточная сеть



Алгоритм Форда-Фалкерсона

1. Обнуляем все потоки. Остаточная сеть изначально совпадает с исходной сетью.
2. В остаточной сети находим любой путь из источника в сток. Если такого пути нет, останавливаемся.
3. Пускаем через найденный путь (*дополняющий путь*) максимально возможный поток:
 1. На найденном пути в остаточной сети ищем ребро с минимальной пропускной способностью *C_{min}*.
 2. Для каждого ребра на найденном пути увеличиваем поток на *C_{min}*, а в противоположном ему — уменьшаем на *C_{min}*.
 3. Модифицируем остаточную сеть. Для всех рёбер на найденном пути, а также для противоположных им рёбер, вычисляем новую пропускную способность. Если она стала ненулевой, добавляем ребро к остаточной сети, а если обнулилась, стираем его.
4. Возвращаемся на шаг 2.



Алгоритм Форда-Фалкерсона

1. Обнуляем все потоки. Остаточная сеть изначально совпадает с исходной сетью.
2. В остаточной сети находим любой путь из источника в сток. Если такого пути нет, останавливаемся.
3. Пускаем через найденный путь (*дополняющий путь*) максимально возможный поток:
 1. На найденном пути в остаточной сети ищем ребро с минимальной пропускной способностью *C_{min}*.
 2. Для каждого ребра на найденном пути увеличиваем поток на *C_{min}*, а в противоположном ему — уменьшаем на *C_{min}*.
 3. Модифицируем остаточную сеть. Для всех рёбер на найденном пути, а также для противоположных им рёбер, вычисляем новую пропускную способность. Если она стала ненулевой, добавляем ребро к остаточной сети, а если обнулилась, стираем его.
4. Возвращаемся на шаг 2.

Асимптотика с целыми пропускными способностями?



Алгоритм Форда-Фалкерсона

1. Обнуляем все потоки. Остаточная сеть изначально совпадает с исходной сетью.
2. В остаточной сети находим любой путь из источника в сток. Если такого пути нет, останавливаемся.
3. Пускаем через найденный путь (*дополняющий путь*) максимально возможный поток:
 1. На найденном пути в остаточной сети ищем ребро с минимальной пропускной способностью *Cmin*.
 2. Для каждого ребра на найденном пути увеличиваем поток на *Cmin*, а в противоположном ему — уменьшаем на *Cmin*.
 3. Модифицируем остаточную сеть. Для всех рёбер на найденном пути, а также для противоположных им рёбер, вычисляем новую пропускную способность. Если она стала ненулевой, добавляем ребро к остаточной сети, а если обнулилась, стираем его.
4. Возвращаемся на шаг 2.

Асимптотика с целыми пропускными способностями?

$O(E \cdot \max f)$

Выполнение
одного шага

Сколько
раз
запустимся



Алгоритм Форда-Фалкерсона

1. Обнуляем все потоки. Остаточная сеть изначально совпадает с исходной сетью.
2. В остаточной сети находим любой путь из источника в сток. Если такого пути нет, останавливаемся.
3. Пускаем через найденный путь (*дополняющий путь*) максимально возможный поток:
 1. На найденном пути в остаточной сети ищем ребро с минимальной пропускной способностью C_{min} .
 2. Для каждого ребра на найденном пути увеличиваем поток на C_{min} , а в противоположном ему — уменьшаем на C_{min} .
 3. Модифицируем остаточную сеть. Для всех рёбер на найденном пути, а также для противоположных им рёбер, вычисляем новую пропускную способность. Если она стала ненулевой, добавляем ребро к остаточной сети, а если обнулилась, стираем его.
4. Возвращаемся на шаг 2.

важно
знать
это при
исходе
из dfs

Асимптотика с целыми пропускными способностями?

$$O(E \cdot \max f)$$

Выполнение
одного шага

Сколько
раз
запустимся

Алгоритм Форда-Фалкерсона

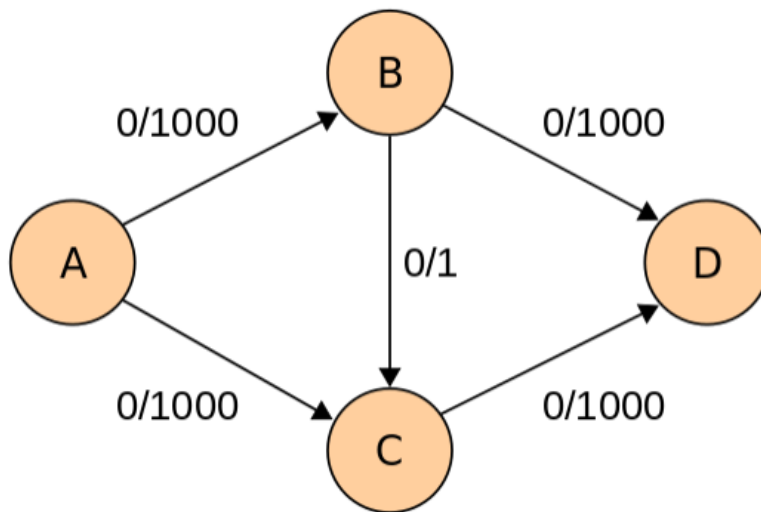
Когда достигается худший случай в алгоритме Форда-Фалкерсона?





Алгоритм Форда-Фалкерсона

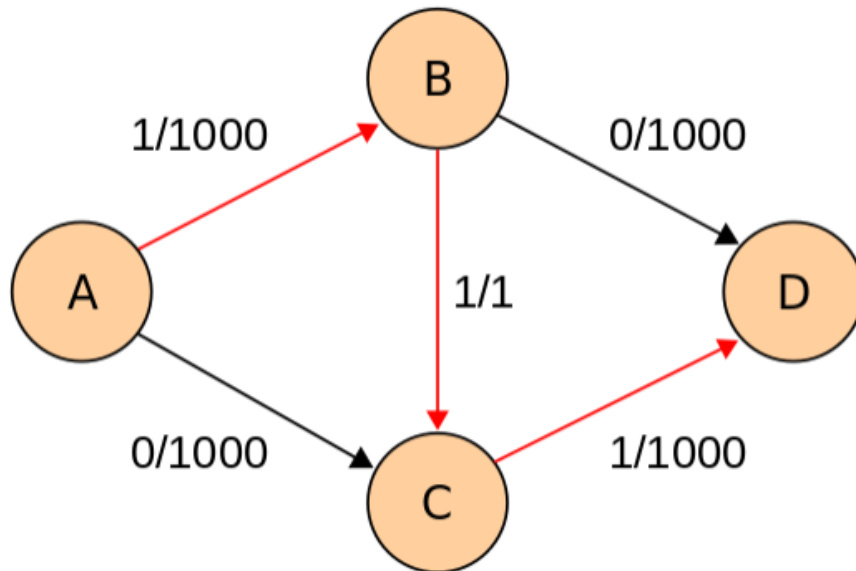
Когда достигается худший случай в алгоритме Форда-Фалкерсона?





Алгоритм Форда-Фалкерсона

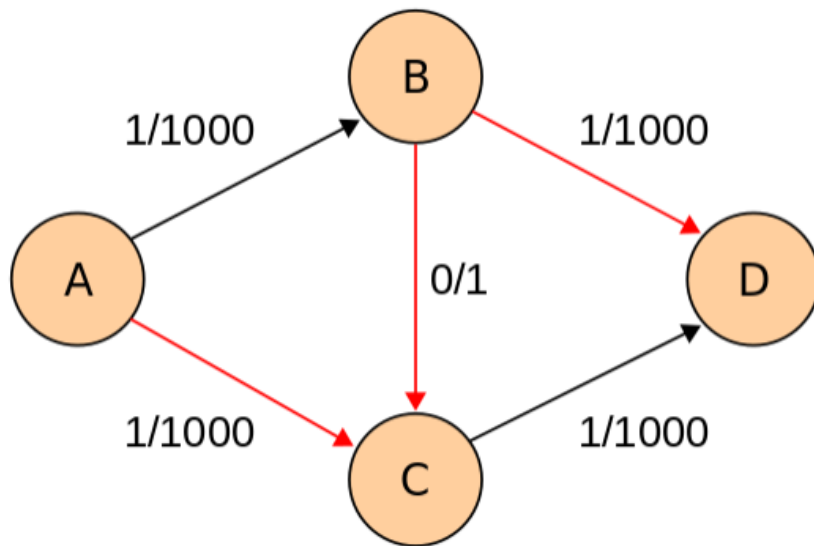
Когда достигается худший случай в алгоритме Форда-Фалкерсона?





Алгоритм Форда-Фалкерсона

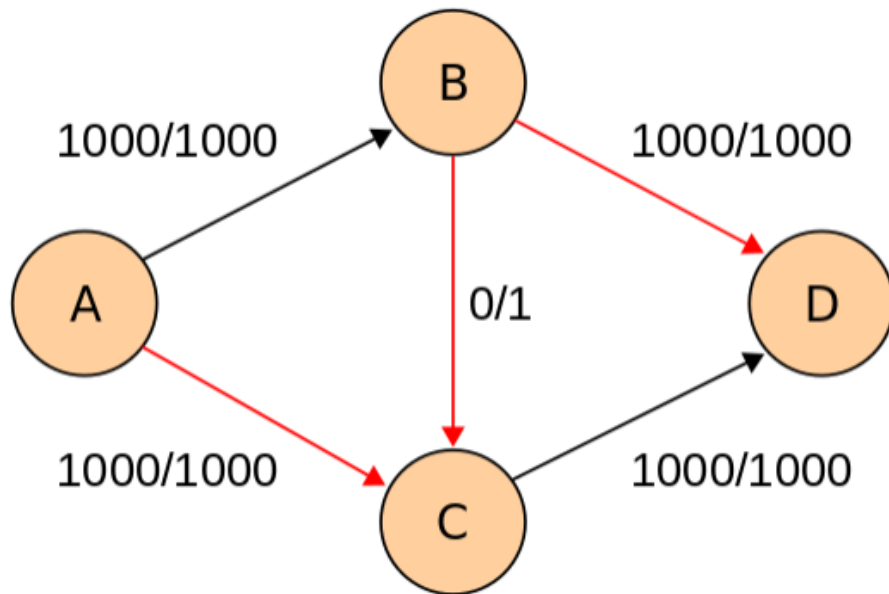
Когда достигается худший случай в алгоритме Форда-Фалкерсона?





Алгоритм Форда-Фалкерсона

Когда достигается худший случай в алгоритме Форда-Фалкерсона?



Алгоритм Эдмондса-Карпа

Давайте выбирать КРАТЧАЙШИЙ по
ребрам из путей из истока в сток



Алгоритм Эдмондса-Карпа

Давайте выбирать КРАТЧАЙШИЙ по
ребрам из путей из истока в сток

Как это можно сделать?



Алгоритм Эдмондса-Карпа

Давайте выбирать КРАТЧАЙШИЙ по
ребрам из путей из истока в сток

Как это можно сделать? BFS!

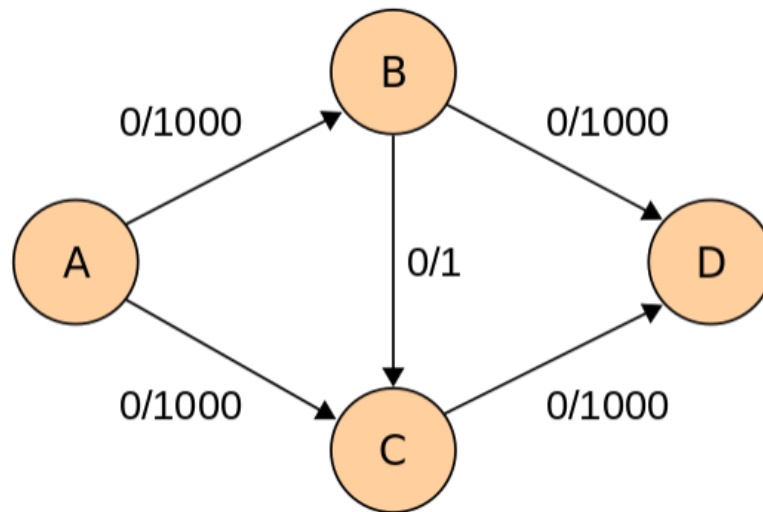




Алгоритм Эдмондса-Карпа

Давайте выбирать КРАТЧАЙШИЙ по ребрам из путей из истока в сток

Как это можно сделать? BFS!

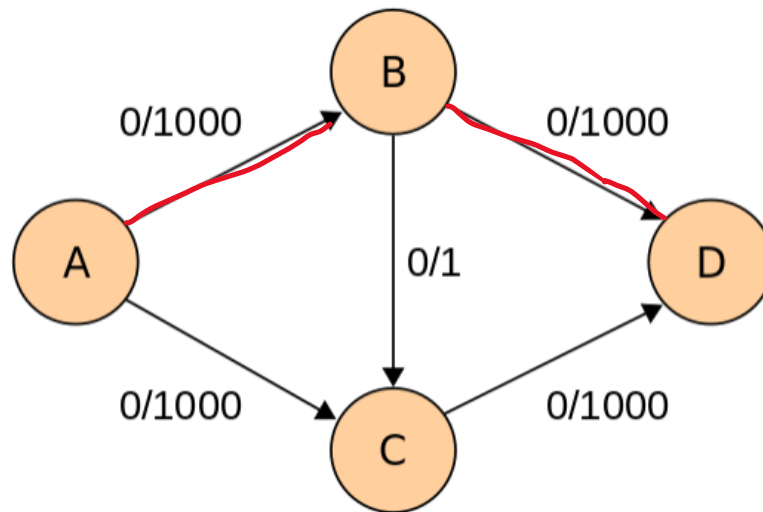




Алгоритм Эдмондса-Карпа

Давайте выбирать КРАТЧАЙШИЙ по ребрам из путей из истока в сток

Как это можно сделать? BFS!

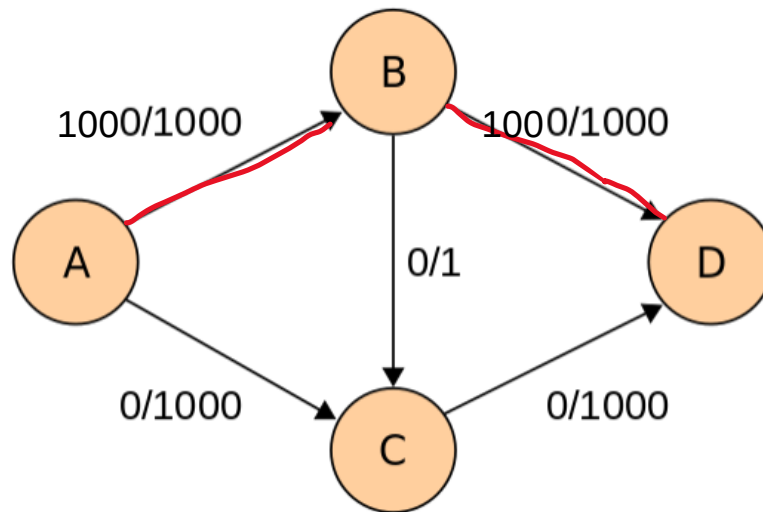




Алгоритм Эдмондса-Карпа

Давайте выбирать КРАТЧАЙШИЙ по ребрам из путей из истока в сток

Как это можно сделать? BFS!

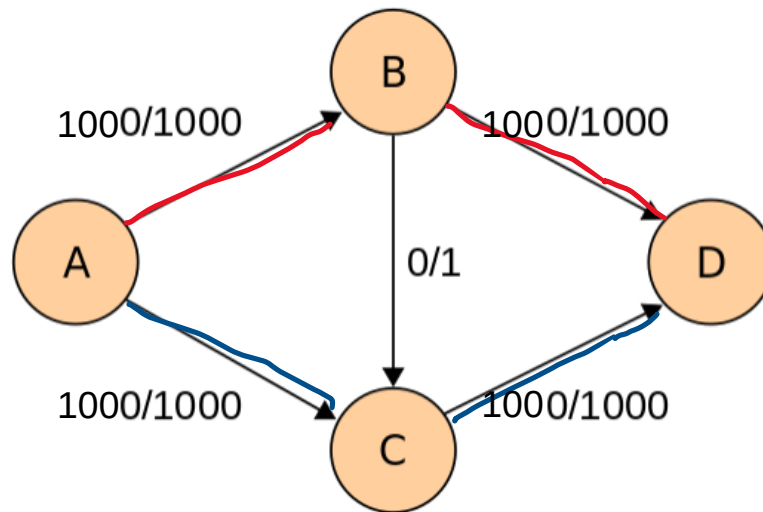




Алгоритм Эдмондса-Карпа

Давайте выбирать КРАТЧАЙШИЙ по ребрам из путей из истока в сток

Как это можно сделать? BFS!



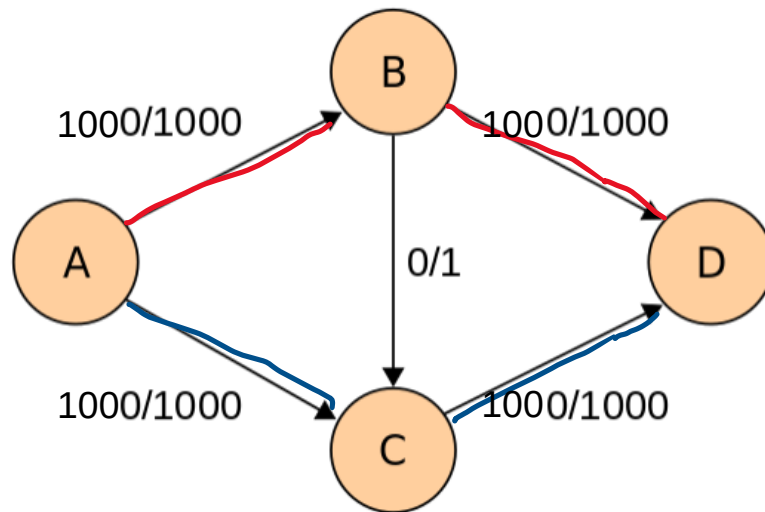


Алгоритм Эдмондса-Карпа

Давайте выбирать КРАТЧАЙШИЙ по ребрам из путей из истока в сток

Как это можно сделать? BFS!

Асимптотика?



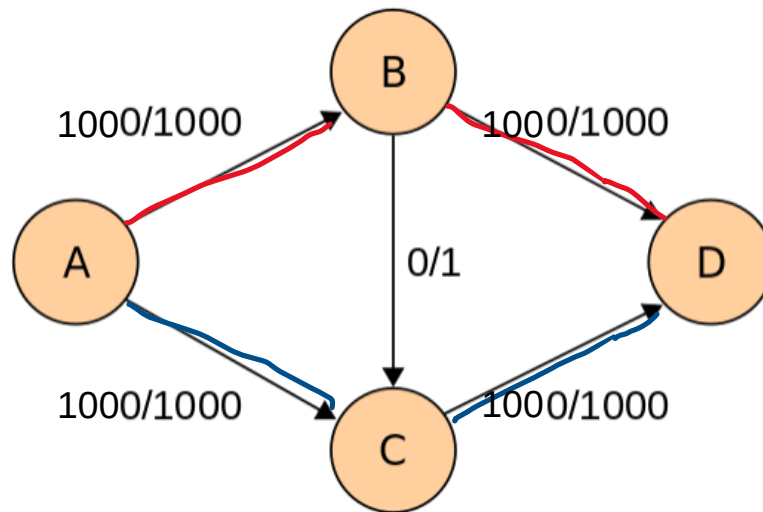


Алгоритм Эдмондса-Карпа

Давайте выбирать КРАТЧАЙШИЙ по ребрам из путей из истока в сток

Как это можно сделать? BFS!

Асимптотика?
 $O(VE^2)$



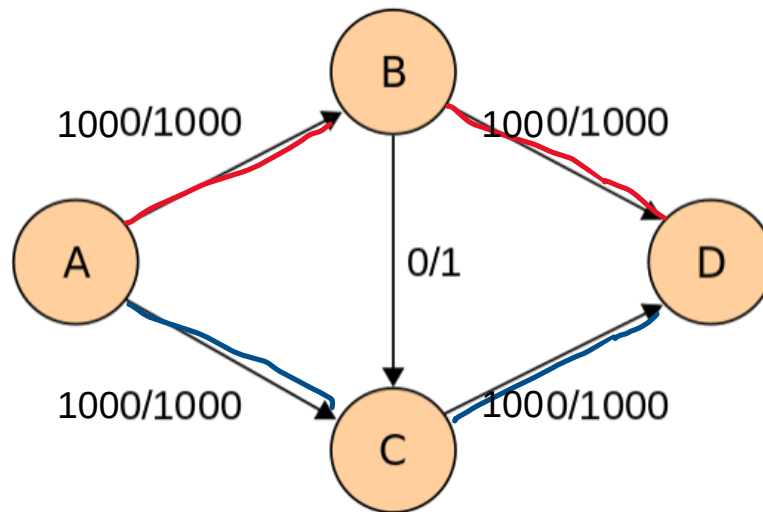


Алгоритм Эдмондса-Карпа. Оптимизация*

Пусть C – пропускная способность
максимального ребра в графе.

Попробуем пропустить по графу
максимальный поток 2^k , такой что
 $2^k < C$.

Если не удалось пропустить такой
поток, уменьшаем k на 1.





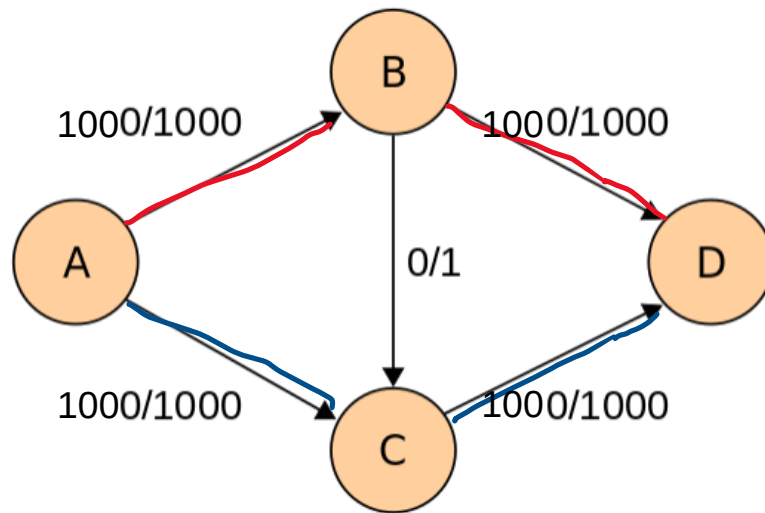
Алгоритм Эдмондса-Карпа. Оптимизация*

Пусть C – пропускная способность
максимального ребра в графе.

Попробуем пропустить по графу
максимальный поток 2^k , такой что
 $2^k < C$.

Если не удалось пропустить такой
поток, уменьшаем k на 1.

Асимптотика?





Алгоритм Эдмондса-Карпа. Оптимизация*

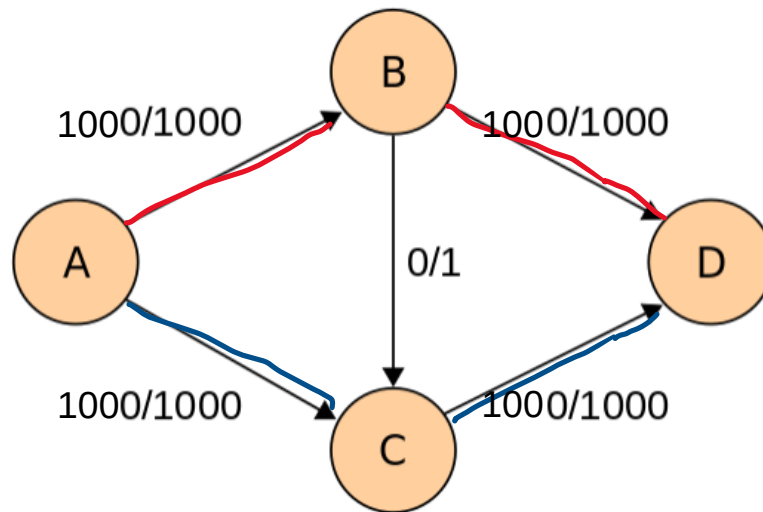
Пусть C – пропускная способность
максимального ребра в графе.

Попробуем пропустить по графу
максимальный поток 2^k , такой что
 $2^k < C$.

Если не удалось пропустить такой
поток, уменьшаем k на 1.

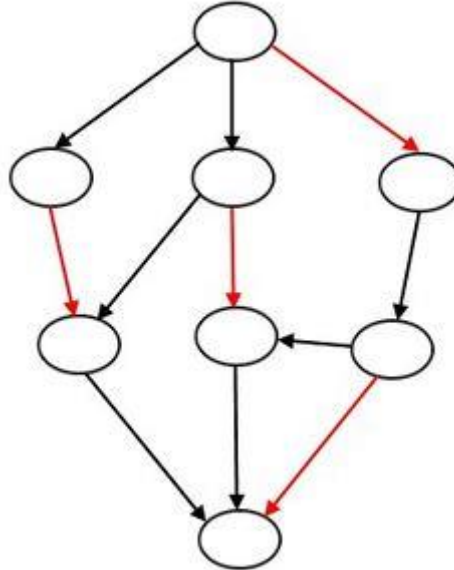
Асимптотика?

$E^2 \log C$



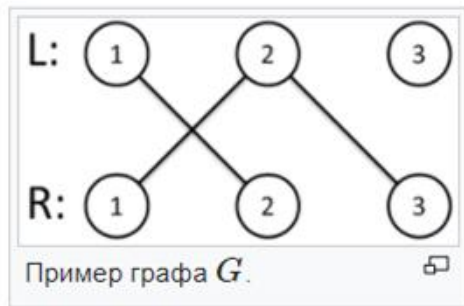
Задача о максимальном паросочетании

Паросочетание (англ. *matching*) M – произвольное множество рёбер графа такое, что никакие два ребра не имеют общей вершины.



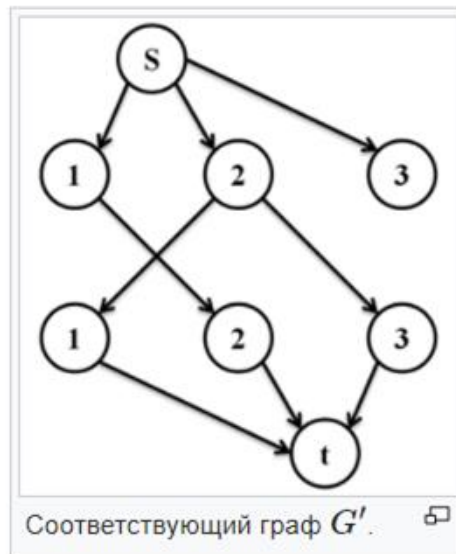
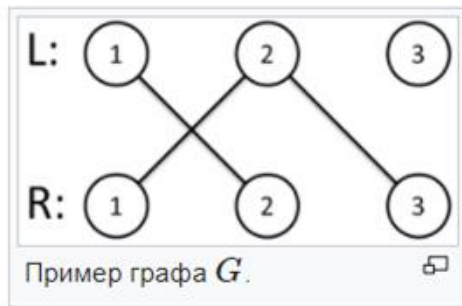
Алгоритм Форда-Факлersona для макс ПарСоч

Вернемся к задаче о поиске максимального паросочетания в двудольном графе.
Попробуем свести задачу к поиску максимального потока.



Алгоритм Форда-Факлersona для макс ПарСоч

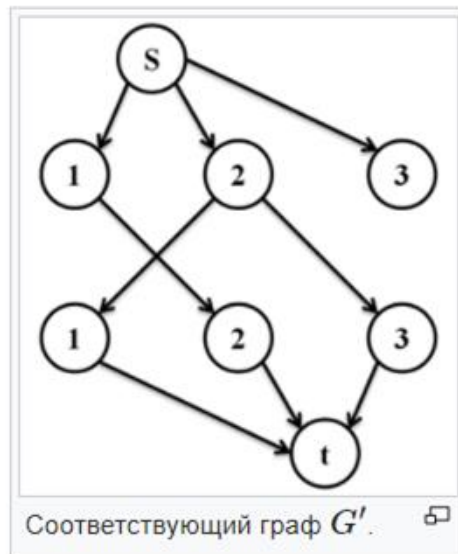
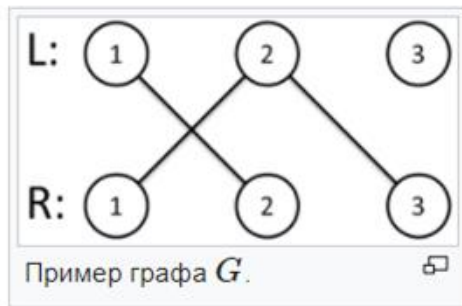
Добавим в граф две вершины. Исток и сток. Из истока добавим ребра во все вершины левой доли, а из правой доли – в сток.



Алгоритм Форда-Факлера для макс ПарСоч

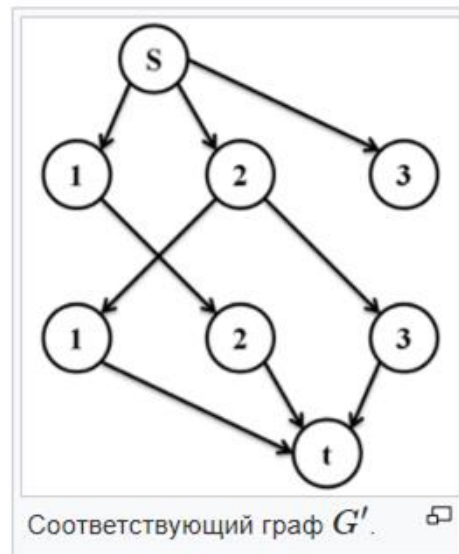
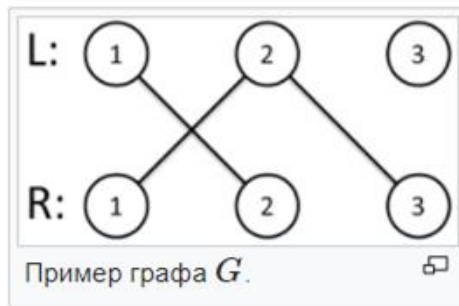
Добавим в граф две вершины. Исток и сток. Из истока добавим ребра во все вершины левой доли, а из правой доли – в сток.

За счет таких вершин сможем гарантировать, что каждая вершина из долей будет использоваться только для одного ребра



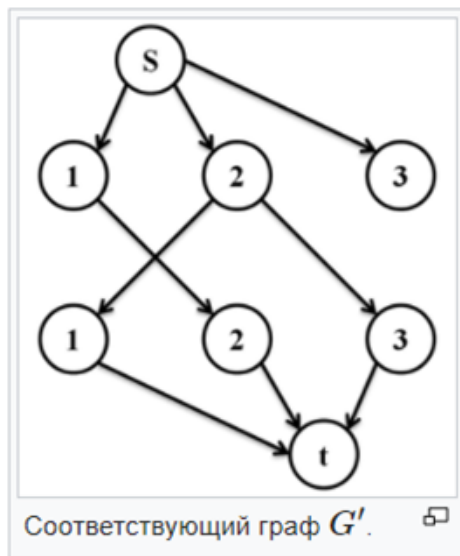
Алгоритм Форда-Факлersona

1. Ищем в графе G' путь из s в t [поиском в глубину](#).
2. Если путь найден, перезаписываем текущее паросочетание. Далее инвертируем все рёбра на пути (ребро (u, v) становится ребром (v, u)) и удаляем (s, L) и (R, t) рёбра, покрывающие вершины, принадлежащие текущему паросочетанию.
3. Если путь не был найден, значит текущее паросочетание является максимальным, и алгоритм завершает работу. Иначе переходим к пункту 1.



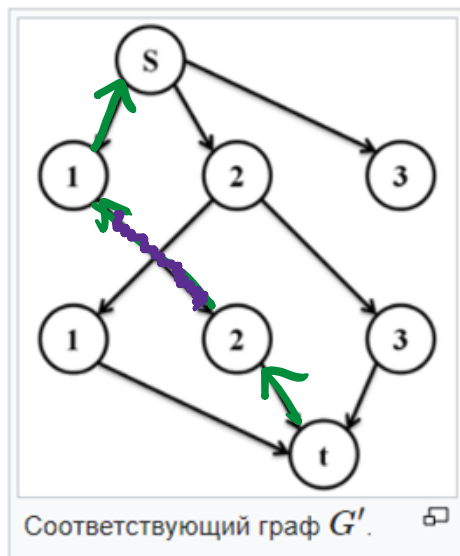
Алгоритм Форда-Факлersonа. Разбор примера

1. Ищем в графе G' путь из s в t [поиском в глубину](#).
2. Если путь найден, перезаписываем текущее паросочетание. Далее инвертируем все рёбра на пути (ребро (u, v) становится ребром (v, u)) и удаляем (s, L) и (R, t) рёбра, покрывающие вершины, принадлежащие текущему паросочетанию.
3. Если путь не был найден, значит текущее паросочетание является максимальным, и алгоритм завершает работу. Иначе переходим к пункту 1.



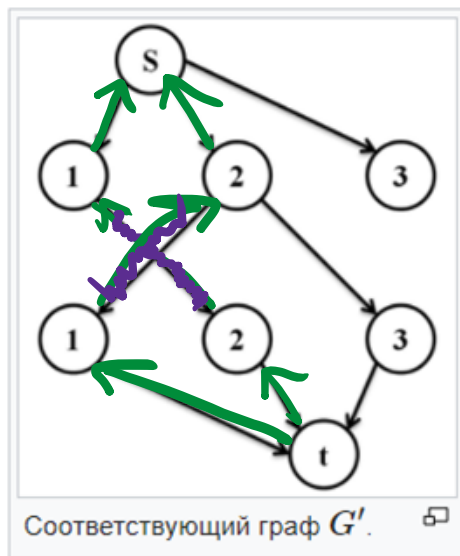
Алгоритм Форда-Факлersonа. Разбор примера

1. Ищем в графе G' путь из s в t **поиском в глубину**.
2. Если путь найден, перезаписываем текущее паросочетание. Далее инвертируем все рёбра на пути (ребро (u, v) становится ребром (v, u)) и удаляем (s, L) и (R, t) рёбра, покрывающие вершины, принадлежащие текущему паросочетанию.
3. Если путь не был найден, значит текущее паросочетание является максимальным, и алгоритм завершает работу. Иначе переходим к пункту 1.



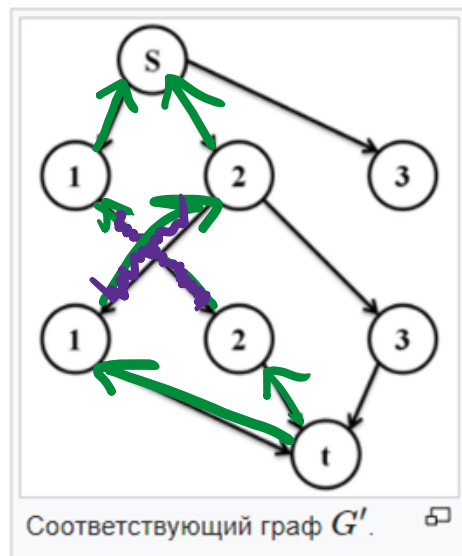
Алгоритм Форда-Факлersonа. Разбор примера

1. Ищем в графе G' путь из s в t **поиском в глубину**.
2. Если путь найден, перезаписываем текущее паросочетание. Далее инвертируем все рёбра на пути (ребро (u, v) становится ребром (v, u)) и удаляем (s, L) и (R, t) рёбра, покрывающие вершины, принадлежащие текущему паросочетанию.
3. Если путь не был найден, значит текущее паросочетание является максимальным, и алгоритм завершает работу. Иначе переходим к пункту 1.



Алгоритм Форда-Факлersona. Разбор примера

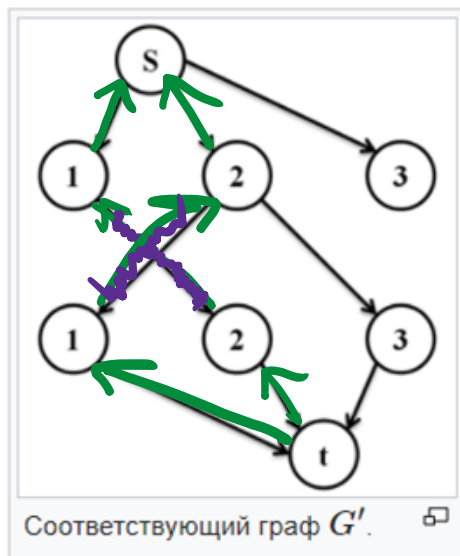
1. Ищем в графе G' путь из s в t поиском в глубину.
2. Если путь найден, перезаписываем текущее паросочетание. Далее инвертируем все рёбра на пути (ребро (u, v) становится ребром (v, u)) и удаляем (s, L) и (R, t) рёбра, покрывающие вершины, принадлежащие текущему паросочетанию.
3. Если путь не был найден, значит текущее паросочетание является максимальным, и алгоритм завершает работу. Иначе переходим к пункту 1.



есть
м
еще
пути?

Алгоритм Форда-Факлersona. Разбор примера

1. Ищем в графе G' путь из s в t поиском в глубину.
2. Если путь найден, перезаписываем текущее паросочетание. Далее инвертируем все рёбра на пути (ребро (u, v) становится ребром (v, u)) и удаляем (s, L) и (R, t) рёбра, покрывающие вершины, принадлежащие текущему паросочетанию.
3. Если путь не был найден, значит текущее паросочетание является максимальным, и алгоритм завершает работу. Иначе переходим к пункту 1.



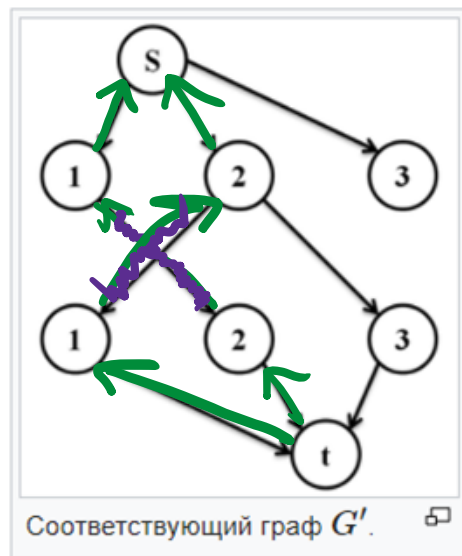
есть
м
еще
путь?

нет: C

Алгоритм Форда-Факлersona. Разбор примера

1. Ищем в графе G' путь из s в t поиском в глубину.
2. Если путь найден, перезаписываем текущее паросочетание. Далее инвертируем все рёбра на пути (ребро (u, v) становится ребром (v, u)) и удаляем (s, L) и (R, t) рёбра, покрывающие вершины, принадлежащие текущему паросочетанию.
3. Если путь не был найден, значит текущее паросочетание является максимальным, и алгоритм завершает работу. Иначе переходим к пункту 1.

тогда
max
паросот
2



есть
м
еще
путь?

нет:с

Алгоритм Форда-Факлersona. Псевдокод

- $px[]$ — массив вершин $y \in R$, инцидентные $x_i \in L$ в текущем паросочетании,
- $py[]$ — массив вершин $x \in L$, инцидентные $y_i \in R$ в текущем паросочетании,
- $vis[]$ — массив, где помечаются посещенные вершины.

Максимальное паросочетание — такие ребра (x, y) , что $x \in L, y \in R, px[x] = y$.

Инициализация и внешний цикл:

```
func fordFulkerson():  
    fill(px, -1)  
    fill(py, -1)  
    isPath = true  
    while isPath  
        isPath = false  
        fill(vis, false)  
        for  $x \in L$   
            if  $px[x] == -1$   
                if dfs(x)  
                    isPath = true
```

проверка
наличие
пути

Поиск в глубину, одновременно инвертирующий ребра:

```
bool dfs(x):  
    if vis[x]  
        return false  
    vis[x] = true  
    for  $(x, y) \in E$   
        if  $py[y] == -1$   
             $py[y] = x$   
             $px[x] = y$   
            return true  
        else  
            if dfs(py[y])  
                 $py[y] = x$   
                 $px[x] = y$   
                return true  
    return false
```

разворот

Алгоритм Форда-Факлersona. Псевдокод

- $px[]$ — массив вершин $y \in R$, инцидентные $x_i \in L$ в текущем паросочетании,
- $py[]$ — массив вершин $x \in L$, инцидентные $y_i \in R$ в текущем паросочетании,
- $vis[]$ — массив, где помечаются посещенные вершины.

Максимальное паросочетание — такие ребра (x, y) , что $x \in L, y \in R, px[x] = y$.

Инициализация и внешний цикл:

```
func fordFulkerson():  
    fill(px, -1)  
    fill(py, -1)  
    isPath = true  
    while isPath  
        isPath = false  
        fill(vis, false)  
        for  $x \in L$   
            if  $px[x] == -1$   
                if dfs(x)  
                    isPath = true
```

Поиск в глубину, одновременно инвертирующий ребра:

```
bool dfs(x):  
    if vis[x]  
        return false  
    vis[x] = true  
    for  $(x, y) \in E$   
        if  $py[y] == -1$   
             $py[y] = x$   
             $px[x] = y$   
            return true  
    else  
        if dfs(py[y])  
             $py[y] = x$   
             $px[x] = y$   
            return true  
    return false
```

Асимптотика?

Алгоритм Форда-Факлersona. Псевдокод

- $px[]$ — массив вершин $y \in R$, инцидентные $x_i \in L$ в текущем паросочетании,
- $py[]$ — массив вершин $x \in L$, инцидентные $y_i \in R$ в текущем паросочетании,
- $vis[]$ — массив, где помечаются посещенные вершины.

Максимальное паросочетание — такие ребра (x, y) , что $x \in L, y \in R, px[x] = y$.

Инициализация и внешний цикл:

```
func fordFulkerson():  
    fill(px, -1)  
    fill(py, -1)  
    isPath = true  
    while isPath  
        isPath = false  
        fill(vis, false)  
        for  $x \in L$   
            if  $px[x] == -1$   
                if dfs(x)  
                    isPath = true
```

Макс размер
паросочетания

DFS

$O(E * V)$

Поиск в глубину, одновременно инвертирующий ребра:

```
bool dfs(x):  
    if vis[x]  
        return false  
    vis[x] = true  
    for  $(x, y) \in E$   
        if  $py[y] == -1$   
             $py[y] = x$   
             $px[x] = y$   
            return true  
    else  
        if dfs(py[y])  
             $py[y] = x$   
             $px[x] = y$   
            return true  
    return false
```

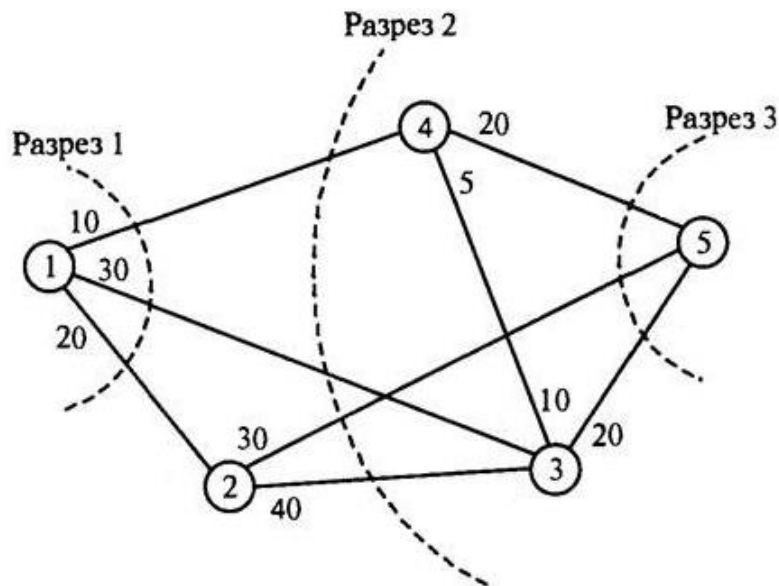
Асимптотика?

Задача о максимальном потоке. Разрез

Определение:

(s, t) -разрезом (англ. *s-t cut*) $\langle S, T \rangle$ в сети G называется пара множеств S, T , удовлетворяющих условиям:

1. $s \in S, t \in T$
2. $S = V \setminus T$



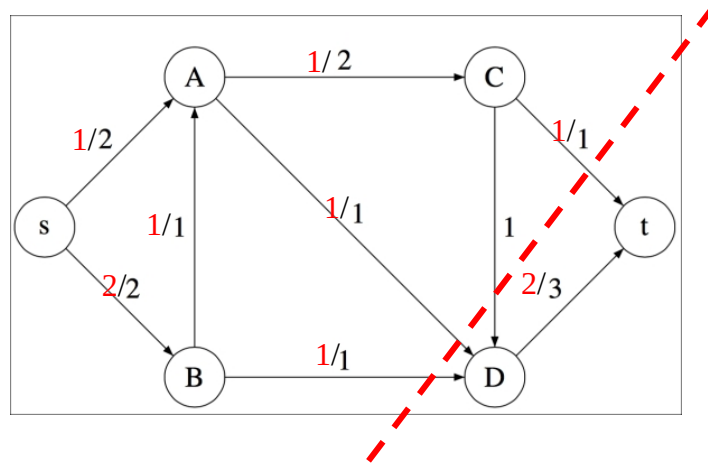
Задача о максимальном потоке. Разрез

Поток через разрез

$$f(S, T) = 3$$

Пропускная способность
разреза

$$c(S, T) = 4$$



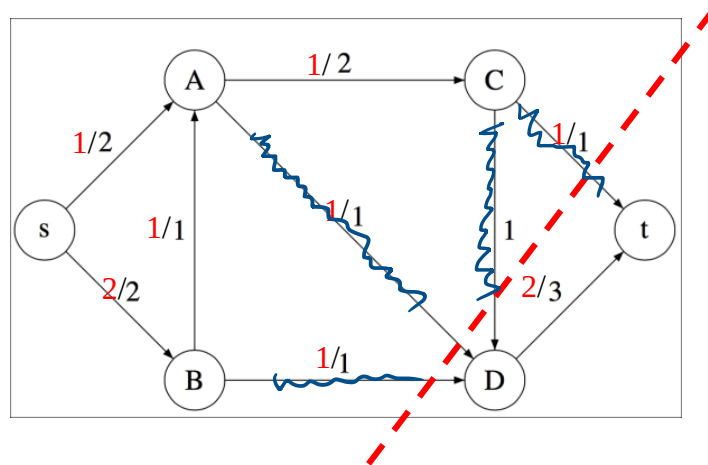
Задача о максимальном потоке. Разрез

Поток через разрез

$$f(S, T) = 3$$

Пропускная способность
разреза

$$c(S, T) = 4$$



Задача о максимальном потоке. Леммы о разрезе

- Минимальный разрез - разрез с минимальной пропускной способностью
- Поток в сети $|f|$ равен потоку разреза $f(S, T)$
- $f(S, T) \leq c(S, T)$ - и для двух **разных** разрезов тоже
- Если $f(S, T) = c(S, T)$, то поток - максимален, а разрез - минимален

